Github Repo:- https://github.com/the-jaypatel/java

Practical 1

Part A

1) Vending machine (VendingMachine.java, main in this file)
(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.
(b) In main, set a snack price of 15 and a running total of 0; create a Scanner.
(c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.
(d) Stop the loop once the total reaches 15 or more.
(e) Print the change to return (total − 15).

VendingMachine.java

Code

```
import java.util.Scanner;
enum Coin { ONE, TWO, FIVE, TEN }
public class VendingMachine {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int price = 15;
        int total = 0;
        while (total < price) {
            System.out.print("Enter coin: ");
            Coin coin = Coin.valueOf(sc.next().toUpperCase());
            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            total += value;
            System.out.println("Total: " + total);
        }
        System.out.println("Paid. Change: " + (total - price));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB01"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>javac VendingMachine.java

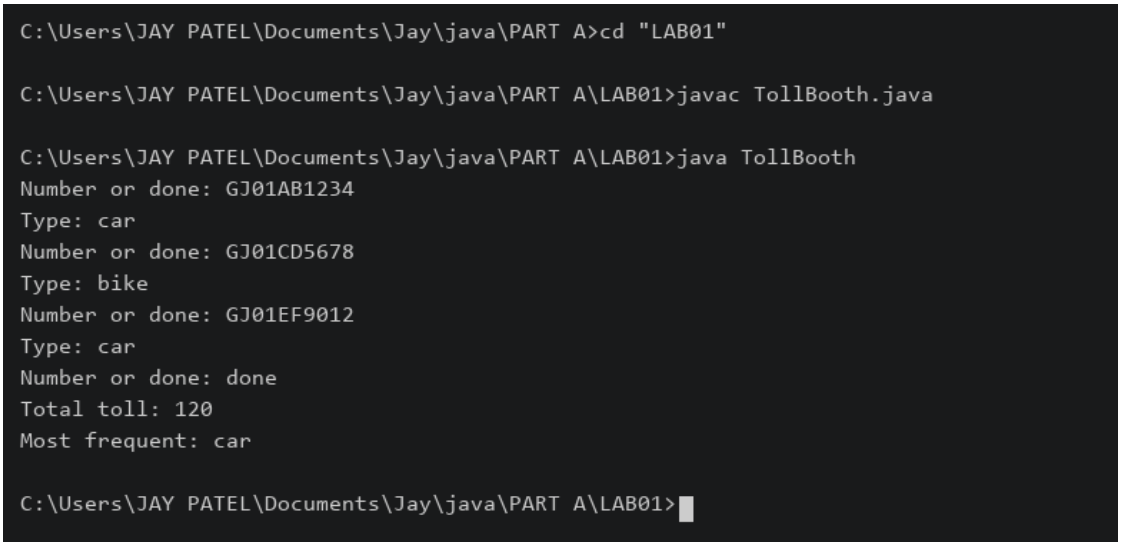
C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>java VendingMachine
Enter coin: FIVE
Total: 5
Enter coin: TEN
Total: 15
Paid. Change: 0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>
```

2) Toll booth (TollBooth.java, main in this file)
(a) Define a record Vehicle(String number, String type).
(b) In main, keep a running toll total and three counters (bike, car, truck).
(c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter.
(d) After the loop, print the total toll and the type with the highest count.

	TollBooth.java

Code	import java.util.Scanner;
	record Vehicle(String number, String type) {}
	public class TollBooth {
	  public static void main(String[] args) {
	  Scanner sc = new Scanner(System.in);
	  int total = 0, bikes = 0, cars = 0, trucks = 0;
	  while (true) {
	  System.out.print("Number or done: ");
	  String number = sc.next();
	  if (number.equalsIgnoreCase("done")) { break; }
	  System.out.print("Type: ");
	  Vehicle v = new Vehicle(number, sc.next().toLowerCase());
	  int toll = switch (v.type()) {
	  case "bike" -> 20;
	  case "car" -> 50;
	  case "truck" -> 150;
	  default -> 0;
	  };
	  switch (v.type()) {
	  case "bike" -> bikes++;
	  case "car" -> cars++;
	  case "truck" -> trucks++;
	  default -> System.out.println("Invalid type");
	  }
	  total += toll;
	  }
	  System.out.println("Total toll: " + total);
	  if (bikes + cars + trucks == 0) {
	  System.out.println("Most frequent: none");
	  } else {
	  int max = Math.max(bikes, Math.max(cars, trucks));
	  if (bikes == max) { System.out.println("Most frequent: bike"); }
	  if (cars == max) { System.out.println("Most frequent: car"); }
	  if (trucks == max) { System.out.println("Most frequent: truck"); }
	  }
	  }
	}

Output	

3)	(For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java) (a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK. (b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases). (c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score. (d) After 5 rounds, print the overall winner.
	RPSLS.java
Code	

```
import java.util.Random;
import java.util.Scanner;
enum Move { ROCK, PAPER, SCISSORS, LIZARD, SPOCK }
public class RPSLS {
   static int winner(Move a, Move b) {
      if (a == b) { return 0; }
      boolean win = switch (a) {
         case ROCK -> b == Move.SCISSORS || b == Move.LIZARD;
         case PAPER -> b == Move.ROCK || b == Move.SPOCK;
         case SCISSORS -> b == Move.PAPER || b == Move.LIZARD;
         case LIZARD -> b == Move.PAPER || b == Move.SPOCK;
         case SPOCK -> b == Move.ROCK || b == Move.SCISSORS;
      };
      if (win) { return 1; }
      return -1;
   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      Random random = new Random();
      int player = 0, computer = 0;
      for (int i = 1; i <= 5; i++) {
         System.out.print("Your move: ");
         Move a = Move.valueOf(sc.next().toUpperCase());
         Move b = Move.values()[random.nextInt(5)];
         System.out.println("You: " + a + " Computer: " + b);
         int result = winner(a, b);
         if (result == 1) { player++; System.out.println("You win round"); }
         else if (result == -1) { computer++; System.out.println("Computer wins round"); }
         else { System.out.println("Tie"); }
      }
      if (player > computer) { System.out.print("You win "); }
      else if (computer > player) { System.out.print("Computer wins "); }
      else { System.out.print("Draw "); }
      System.out.println(player + "-" + computer);
   }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB01"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>javac RPSLS.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>java RPSLS
Your move: ROCK
You: ROCK Computer: SPOCK
Computer wins round
Your move: PAPER
You: PAPER Computer: LIZARD
Computer wins round
Your move: SCISSORS
You: SCISSORS Computer: LIZARD
You win round
Your move: LIZARD
You: LIZARD Computer: LIZARD
Tie
Your move: SPOCK
You: SPOCK Computer: SPOCK
Tie
Computer wins 1-2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB01>
```

Practical 2

Part A	
1)	Smart thermostat (Thermostat.java, main in this file) (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print “Already at maximum (30)”. (e) lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”. (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().
	Thermostat.java
Code	

```
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16, MAX = 30;
    private static int activeCount = 0;
    Thermostat(String location, int startTemp) {
        this.location = location;
        temperature = 22;
        if (startTemp >= MIN && startTemp <= MAX) { temperature = startTemp; }
        activeCount++;
    }
    Thermostat(String location) { this(location, 22); }
    void raise() {
        if (temperature < MAX) { temperature++; }
        else { System.out.println("Already at maximum (30)"); }
    }
    void lower() {
        if (temperature > MIN) { temperature--; }
        else { System.out.println("Already at minimum (16)"); }
    }
    int getTemperature() { return temperature; }
    static int getActiveCount() { return activeCount; }
    public static void main(String[] args) {
        Thermostat a = new Thermostat("Room 1");
        Thermostat b = new Thermostat("Room 2", 24);
        for (int i = 0; i < 10; i++) {
            a.raise(); System.out.println(a.getTemperature());
        }
        for (int i = 0; i < 20; i++) {
            a.lower(); System.out.println(a.getTemperature());
        }
        System.out.println("Active: " + getActiveCount());
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB02"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>javac Thermostat.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>java Thermostat
23
24
25
26
27
28
29
30
Already at maximum (30)
30
Already at maximum (30)
30
29
28
27
26
25
24
23
22
21
20
19
18
17
16
Already at minimum (16)
16
Already at minimum (16)
16
Already at minimum (16)
16
Already at minimum (16)
16
Already at minimum (16)
16
Already at minimum (16)
16
Active: 2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>
```

2)	Cinema show (CinemaShow.java, main in this file) (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().

	CinemaShow.java

Code	public class CinemaShow {
	private String title;
	private int seatsAvailable;
	private final int capacity;
	private static int totalBooked = 0;
	CinemaShow(String title, int capacity) {
	this.title = title;
	this.capacity = capacity;
	seatsAvailable = capacity;
	}
	CinemaShow(String title) { this(title, 100); }
	boolean book(int n) {
	if (n > 0 && n <= seatsAvailable) {
	seatsAvailable -= n; totalBooked += n; return true;
	}
	return false;
	}
	void cancel(int n) {
	if (n > 0) { seatsAvailable = Math.min(capacity, seatsAvailable + n); }
	}
	int getSeatsAvailable() { return seatsAvailable; }
	static int getTotalBooked() { return totalBooked; }
	public static void main(String[] args) {
	CinemaShow s = new CinemaShow("Movie", 10);
	System.out.println(s.book(6) + " Seats: " + s.getSeatsAvailable());
	System.out.println(s.book(8) + " Seats: " + s.getSeatsAvailable());
	s.cancel(2);
	System.out.println("After cancel: " + s.getSeatsAvailable());
	System.out.println(s.book(3) + " Seats: " + s.getSeatsAvailable());
	System.out.println("Total booked: " + getTotalBooked());
	}
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB02" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>javac CinemaShow.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>java CinemaShow true Seats: 4 false Seats: 4 After cancel: 6 true Seats: 3 Total booked: 9 C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>

3)	(For Advanced Learners – home) Parking lot (ParkingLot.java) (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0. (b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject. (c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.
	ParkingLot.java
Code	public class ParkingLot { private int twoWheelers, fourWheelers; private final int twoCap, fourCap; private static long revenue = 0;

```
    ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap; this.fourCap = fourCap;
    }
    void park(String type) {
        if (type.equals("two") && twoWheelers < twoCap) {
            twoWheelers++; revenue += 20; System.out.println("Two parked");
        } else if (type.equals("four") && fourWheelers < fourCap) {
            fourWheelers++; revenue += 40; System.out.println("Four parked");
        } else { System.out.println("Full or invalid type"); }
    }
    void leave(String type) {
        if (type.equals("two") && twoWheelers > 0) { twoWheelers--; }
        else if (type.equals("four") && fourWheelers > 0) { fourWheelers--; }
        System.out.println("Occupancy: " + twoWheelers + ", " + fourWheelers);
    }
    public static void main(String[] args) {
        ParkingLot p = new ParkingLot(1, 1);
        p.park("two"); p.park("two"); p.park("four");
        p.leave("two"); p.park("two");
        System.out.println("Final: " + p.twoWheelers + ", " + p.fourWheelers);
        System.out.println("Revenue: " + revenue);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB02"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>javac ParkingLot.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>java ParkingLot
Two parked
Full or invalid type
Four parked
Occupancy: 0, 1
Two parked
Final: 1, 1
Revenue: 80

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB02>
```

Practical 3

Part A	
1)	Distinct points (Point.java + Driver.java with main) (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return “(x, y)”. (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print “Distinct: N”.
	Point.java

Code	`import java.util.Objects;`
	`public class Point {`
	`  private int x, y;`
	`  Point(int x, int y) { this.x = x; this.y = y; }`
	`  public String toString() { return "(" + x + ", " + y + ")"; }`
	`  public boolean equals(Object o) {`
	`  if (!(o instanceof Point p)) { return false; }`
	`  return x == p.x && y == p.y;`
	`  }`
	`  public int hashCode() { return Objects.hash(x, y); }`
	`}`

	Driver.java

Code	`public class Driver {`
	`  public static void main(String[] args) {`
	`  Point[] points = {new Point(1,2), new Point(3,4), new Point(1,2),`
	`  new Point(5,6), new Point(3,4)};`
	`  int count = 0;`
	`  for (int i = 0; i < points.length; i++) {`
	`  boolean found = false;`
	`  for (int j = 0; j < i; j++) {`
	`  if (points[i].equals(points[j])) { found = true; break; }`
	`  }`
	`  if (!found) { count++; }`
	`  }`
	`  System.out.println("Distinct: " + count);`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB03\Q1"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q1>javac Point.java Driver.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q1>java Driver`
	`Distinct: 3`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q1>`

2)	Duplicate card check (Card.java + Driver.java) (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: <card>" the first time a repeat appears.
	Card.java

Code	`import java.util.Objects;`
	`public class Card {`
	`  private String rank, suit;`
	`  Card(String rank, String suit) { this.rank = rank; this.suit = suit; }`
	`  public String toString() { return rank + " of " + suit; }`
	`  public boolean equals(Object o) {`
	`  if (!(o instanceof Card c)) { return false; }`
	`  return rank.equals(c.rank) && suit.equals(c.suit);`
	`  }`
	`  public int hashCode() { return Objects.hash(rank, suit); }`
	`}`

	Driver.java

Code	`public class Driver {`
	`  public static void main(String[] args) {`
	`  Card[] input = {new Card("Ace", "Spades"), new Card("Queen", "Hearts"),`
	`  new Card("Ace", "Spades")};`
	`  Card[] cards = new Card[input.length];`
	`  int size = 0;`
	`  for (Card c : input) {`
	`  boolean duplicate = false;`
	`  for (int i = 0; i < size; i++) {`
	`  if (c.equals(cards[i])) { duplicate = true; break; }`
	`  }`
	`  if (duplicate) {`
	`  System.out.println("Duplicate found: " + c); break;`
	`  }`
	`  cards[size++] = c;`
	`  }`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB03\Q2"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q2>javac Card.java Driver.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q2>java Driver`
	`Duplicate found: Ace of Spades`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q2>`

3)	(For Advanced Learners – home) Fractions (Fraction.java + Driver.java) (a) Create class Fraction with private int num, den. (b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g. (c) Override toString() to return num + "/" + den. (d) Override equals()/hashCode() using the reduced num and den. (e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.

	Fraction.java

Code	import java.util.Objects;
	public class Fraction {
	  private int num, den;
	  Fraction(int num, int den) {
	  if (den == 0) { throw new IllegalArgumentException("Zero denominator"); }
	  if (den < 0) { num = -num; den = -den; }
	  int g = gcd(Math.abs(num), den);
	  this.num = num / g; this.den = den / g;
	  }
	  private int gcd(int a, int b) {
	  while (b != 0) { int t = a % b; a = b; b = t; }
	  return a;
	  }
	  public String toString() { return num + "/" + den; }
	  public boolean equals(Object o) {
	  if (!(o instanceof Fraction f)) { return false; }
	  return num == f.num && den == f.den;
	  }
	  public int hashCode() { return Objects.hash(num, den); }
	}

	Driver.java

Code	public class Driver {
	  public static void main(String[] args) {
	  Fraction a = new Fraction(1, 2);
	  Fraction b = new Fraction(2, 4);
	  Fraction c = new Fraction(3, 6);
	  System.out.println(a + " " + b + " " + c);
	  System.out.println(a.equals(b) && b.equals(c));
	  }
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB03\Q3" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q3>javac Fraction.java Driver.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q3>java Driver 1/2 1/2 1/2 true C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB03\Q3>

Practical 4

Part A

1) Password strength (PasswordChecker.java + Driver.java)
(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.
(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4).
(c) In Driver.main: test several passwords; for each, print which rules passed and the final label.

PasswordChecker.java

Code

```
public class PasswordChecker {
    static boolean[] rules(String pw) {
        return new boolean[]{pw.length() >= 8, pw.matches(".*[A-Z].*"),
            pw.matches(".*[0-9].*"), pw.matches(".*[^a-zA-Z0-9].*")};
    }
    static String strength(String pw) {
        int count = 0;
        for (boolean pass : rules(pw)) { if (pass) { count++; } }
        if (count <= 1) { return "Weak"; }
        if (count <= 3) { return "Medium"; }
        return "Strong";
    }
}
```

Driver.java

Code

```
import java.util.Arrays;
public class Driver {
    public static void main(String[] args) {
        String[] passwords = {"abc", "Abcd1234!", "abcd1234"};
        for (String pw : passwords) {
            System.out.println(pw);
            System.out.println("Length, uppercase, digit, special: "
                + Arrays.toString(PasswordChecker.rules(pw)));
            System.out.println(PasswordChecker.strength(pw));
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB04\Q1"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q1>javac PasswordChecker.java Driver.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q1>java Driver
abc
Length, uppercase, digit, special: [false, false, false, false]
Weak
Abcd1234!
Length, uppercase, digit, special: [true, true, true, true]
Strong
abcd1234
Length, uppercase, digit, special: [true, false, true, false]
Medium

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q1>
```

2)	Chat-log filter (ChatFilter.java + Driver.java) (a) In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts). (b) Read a keyword from the user. (c) For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts. (d) If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder. (e) Print “Matches: N” and the built report.
	ChatFilter.java

Code	`public class ChatFilter {`
	`  static String filter(String[] lines, String keyword) {`
	`  int count = 0;`
	`  StringBuilder report = new StringBuilder();`
	`  for (String line : lines) {`
	`  String[] p = line.split(" ", 3);`
	`  if (p.length != 3) { continue; }`
	`  if (p[2].toLowerCase().contains(keyword.toLowerCase())) {`
	`  count++;`
	`  report.append(p[0]).append(" ").append(p[1]).append(": ")`
	`  .append(p[2]).append("\n");`
	`  }`
	`  }`
	`  return "Matches: " + count + "\n" + report;`
	`  }`
	`}`

	Driver.java

Code	`import java.util.Scanner;`
	`public class Driver {`
	`  public static void main(String[] args) {`
	`  String[] lines = {"10:05 alice Hello there", "10:06 bob Good morning", "bad"};`
	`  System.out.print("Keyword: ");`
	`  String keyword = new Scanner(System.in).nextLine();`
	`  System.out.print(ChatFilter.filter(lines, keyword));`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB04\Q2"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q2>javac ChatFilter.java Driver.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q2>java Driver`
	`Keyword: hello`
	`Matches: 1`
	`10:05 alice: Hello there`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB04\Q2>`

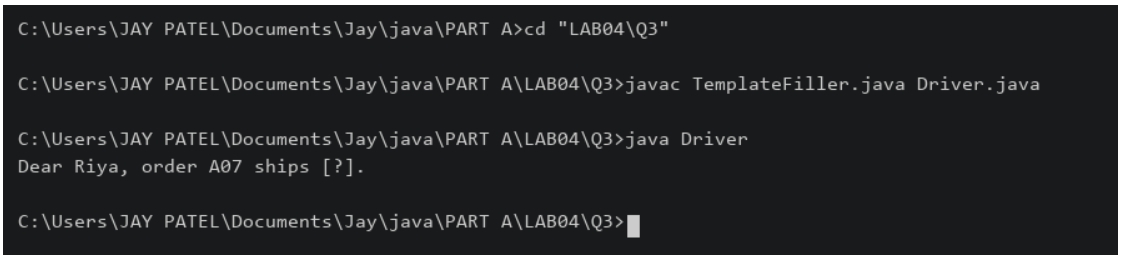
3)	(For Advanced Learners – home) Template filler (TemplateFiller.java + Driver.java) (a) In Driver.main, define a template like “Dear {name}, order {id} ships {date}.” and two parallel arrays: names {“name”, “id”} and values {“Riya”, “A07”} (note {date} is not supplied). (b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template. (c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with “[?]” if not found. (d) Print the filled template.

	TemplateFiller.java

Code	import java.util.regex.*;
	public class TemplateFiller {
	  static String fill(String template, String[] names, String[] values) {
	  Matcher m = Pattern.compile("\\{(\\w+)\\}").matcher(template);
	  StringBuilder result = new StringBuilder();
	  while (m.find()) {
	  String value = "[?]";
	  for (int i = 0; i < names.length; i++) {
	  if (names[i].equals(m.group(1))) { value = values[i]; break; }
	  }
	  m.appendReplacement(result, Matcher.quoteReplacement(value));
	  }
	  m.appendTail(result);
	  return result.toString();
	  }
	}

	Driver.java

Code	public class Driver {
	  public static void main(String[] args) {
	  String template = "Dear {name}, order {id} ships {date}.";
	  String[] names = {"name", "id"};
	  String[] values = {"Riya", "A07"};
	  System.out.println(TemplateFiller.fill(template, names, values));
	  }
	}

Output	

Practical 5

Part A	
1)	Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism.
	ShapeDemo.java

Code	abstract class Shape { abstract double area(); }
	class Circle extends Shape {
	double radius;
	Circle(double radius) { this.radius = radius; }
	double area() { return Math.PI * radius * radius; }
	}
	class Rectangle extends Shape {
	double length, width;
	Rectangle(double length, double width) { this.length = length; this.width = width; }
	double area() { return length * width; }
	}
	class Triangle extends Shape {
	double base, height;
	Triangle(double base, double height) { this.base = base; this.height = height; }
	double area() { return base * height / 2; }
	}
	public class ShapeDemo {
	public static void main(String[] args) {
	Shape[] shapes = {new Circle(2), new Rectangle(4,5), new Triangle(3,6)};
	double total = 0, largest = -1;
	String name = "";
	for (Shape s : shapes) {
	double area = s.area(); total += area;
	System.out.println(s.getClass().getSimpleName() + ": " + area
	+ " Running total: " + total);
	if (area > largest) { largest = area; name = s.getClass().getSimpleName(); }
	}
	System.out.println("Largest: " + name + " " + largest);
	}
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB05" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>javac ShapeDemo.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>java ShapeDemo Circle: 12.566370614359172 Running total: 12.566370614359172 Rectangle: 20.0 Running total: 32.56637061435917 Triangle: 9.0 Running total: 41.56637061435917 Largest: Rectangle 20.0 C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>

2)	Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.
	Payroll.java

Code	abstract class Employee {
	String name, id;
	Employee(String name, String id) { this.name = name; this.id = id; }
	abstract double monthlySalary();

```
}
class FullTime extends Employee {
    double salary;
    FullTime(String n, String id, double salary) { super(n,id); this.salary = salary; }
    double monthlySalary() { return salary; }
}
class PartTime extends Employee {
    int hours; double rate;
    PartTime(String n, String id, int hours, double rate) {
        super(n,id); this.hours = hours; this.rate = rate;
    }
    double monthlySalary() { return hours * rate; }
}
class Intern extends Employee {
    double stipend;
    Intern(String n, String id, double stipend) { super(n,id); this.stipend = stipend; }
    double monthlySalary() { return stipend; }
}
public class Payroll {
    public static void main(String[] args) {
        Employee[] staff = {new FullTime("Tej","E1",30000),
            new PartTime("Riya","E2",40,200), new Intern("Aman","E3",5000)};
        double total = 0;
        for (Employee e : staff) {
            System.out.println(e.id + " " + e.name + " " + e.monthlySalary());
            total += e.monthlySalary();
            if (e instanceof Intern) { System.out.println("Intern stipend"); }
        }
        System.out.println("Total: " + total);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB05"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>javac Payroll.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>java Payroll
E1 Tej 30000.0
E2 Riya 8000.0
E3 Aman 5000.0
Intern stipend
Total: 43000.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>
```

3) (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

MediaDemo.java

Code

```
abstract class Media {
    String title; int daysLate;
    Media(String title, int daysLate) { this.title = title; this.daysLate = daysLate; }
    abstract double lateFee();
}
class Book extends Media {
    Book(String title, int days) { super(title, days); }
    double lateFee() { return Math.max(0, daysLate) * 2; }
}
class DVD extends Media {
    DVD(String title, int days) { super(title, days); }
    double lateFee() { return Math.max(0, daysLate) * 5; }
}
class Magazine extends Media {
    Magazine(String title, int days) { super(title, days); }
    double lateFee() { return Math.max(0, daysLate - 1); }
```

```
}
public class MediaDemo {
    public static void main(String[] args) {
        Media[] items = {new Book("Java",3), new DVD("Movie",2),
                         new Magazine("Science",4)};
        double total = 0;
        for (Media m : items) {
            System.out.println(m.title + ": " + m.lateFee()); total += m.lateFee();
        }
        System.out.println("Total fees: " + total);
    }
}
```

Rules: Book Rs 2/day; DVD Rs 5/day; Magazine one free day, then Rs 1/day.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB05"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>javac MediaDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>java MediaDemo
Java: 6.0
Movie: 10.0
Science: 3.0
Total fees: 19.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB05>
```

Practical 6

Part A	
1)	Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.
	RemoteDemo.java

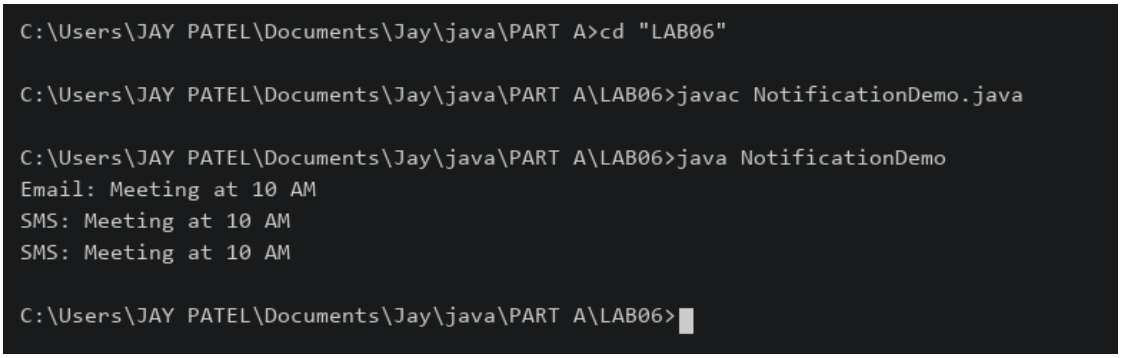
Code	interface Switchable {
	  void on();
	  void off();
	  default void toggle() { on(); off(); }
	}
	class Fan implements Switchable {
	  public void on() { System.out.println("Fan ON"); }
	  public void off() { System.out.println("Fan OFF"); }
	}
	class Light implements Switchable {
	  public void on() { System.out.println("Light ON"); }
	  public void off() { System.out.println("Light OFF"); }
	}
	@FunctionalInterface
	interface Permission { boolean allow(Switchable device, int hour); }
	public class RemoteDemo {
	  public static void main(String[] args) {
	  Switchable[] devices = {new Fan(), new Light()};
	  for (Switchable d : devices) { d.toggle(); }
	  Permission p1 = new Permission() {
	  public boolean allow(Switchable d, int hour) { return hour < 22; }
	  };
	  Permission p2 = (d, hour) -> hour < 22;
	  System.out.println(p1.allow(devices[0], 10));
	  System.out.println(p2.allow(devices[0], 10));
	  }
	}

Output	C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB06" C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>javac RemoteDemo.java C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>java RemoteDemo Fan ON Fan OFF Light ON Light OFF true true C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06>

2)	Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.
	NotificationDemo.java
Code	@FunctionalInterface interface Notifier { void send(String message); } interface Urgent {} public class NotificationDemo {

```
    public static void main(String[] args) {
        Notifier email = message -> System.out.println("Email: " + message);
        Notifier sms = (Notifier & Urgent) message -> System.out.println("SMS: " + message);
        Notifier[] senders = {email, sms};
        for (Notifier sender : senders) {
            sender.send("Meeting at 10 AM");
            if (sender instanceof Urgent) { sender.send("Meeting at 10 AM"); }
        }
    }
}
```

Output



3) (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.

rules/Discount.java

Code

```
package rules;
@FunctionalInterface
public interface Discount { double apply(double price); }
```

app/DiscountApp.java

Code

```
package app;
import java.util.Scanner;
import rules.Discount;
public class DiscountApp {
    public static void main(String[] args) {
        double[] prices = {100, 200, 500};
        Discount[] rules = {p -> p, p -> p * 0.9, p -> p * 0.8};
        System.out.println("1. No discount\n2. 10% discount\n3. 20% discount");
        int choice = new Scanner(System.in).nextInt();
        if (choice < 1 || choice > 3) { System.out.println("Invalid choice"); return; }
        Discount selected = rules[choice - 1];
        for (double p : prices) { System.out.println(p + " -> " + selected.apply(p)); }
    }
}
```

Build and run from the folder containing app and rules:
javac -d out rules/Discount.java app/DiscountApp.java
jar --create --file discount.jar --main-class app.DiscountApp -C out .
java -jar discount.jar

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB06\Q3"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>javac -d out rules\Discount.java app\DiscountApp.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>jar --create --file discount.jar --main-class app.DiscountApp -C out .

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>java -jar discount.jar
1. No discount
2. 10% discount
3. 20% discount
2
100.0 -> 90.0
200.0 -> 180.0
500.0 -> 450.0

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB06\Q3>
```

Practical 7

Part A	
1)	Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.
	FormDemo.java
Code	

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength { int value(); }
class SignupForm {
    @NotBlank @MaxLength(10) String name;
    @NotBlank String email;
    SignupForm(String name, String email) { this.name = name; this.email = email; }
}
public class FormDemo {
    static ArrayList<String> check(Object obj) throws IllegalAccessException {
        ArrayList<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object value = f.get(obj);
            if (f.isAnnotationPresent(NotBlank.class)
                && (value == null || value.toString().isBlank())) {
                errors.add(f.getName() + " must not be blank");
            }
            MaxLength rule = f.getAnnotation(MaxLength.class);
            if (rule != null && value != null && value.toString().length() > rule.value()) {
                errors.add(f.getName() + " is too long");
            }
        }
        return errors;
    }
    public static void main(String[] args) throws Exception {
        System.out.println(check(new SignupForm("VeryLongUserName", "")));
        System.out.println(check(new SignupForm("Tej", "tej@test.com")));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB07"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>javac FormDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>java FormDemo
[name is too long, email must not be blank]
[]

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>
```

2)	Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.
	MiniRunner.java
Code	import java.lang.annotation.*;

```
import java.lang.reflect.*;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}
class Checks {
   @Run public void first() { System.out.println("First ran"); }
   @Run public void second() { System.out.println("Second ran"); }
   public void helper() { System.out.println("Helper"); }
}
public class MiniRunner {
   public static void main(String[] args) throws Exception {
      Checks checks = new Checks();
      int count = 0;
      for (Method m : Checks.class.getDeclaredMethods()) {
         if (m.isAnnotationPresent(Run.class) && m.getParameterCount() == 0) {
            m.setAccessible(true);
            try { m.invoke(checks); }
            catch (InvocationTargetException e) {
               System.out.println("Failed: " + e.getCause().getMessage());
            }
            count++;
         }
      }
      System.out.println("Methods ran: " + count);
   }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB07"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>javac MiniRunner.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>java MiniRunner
First ran
Second ran
Methods ran: 2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB07>
```

3) (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

ColumnDemo.java

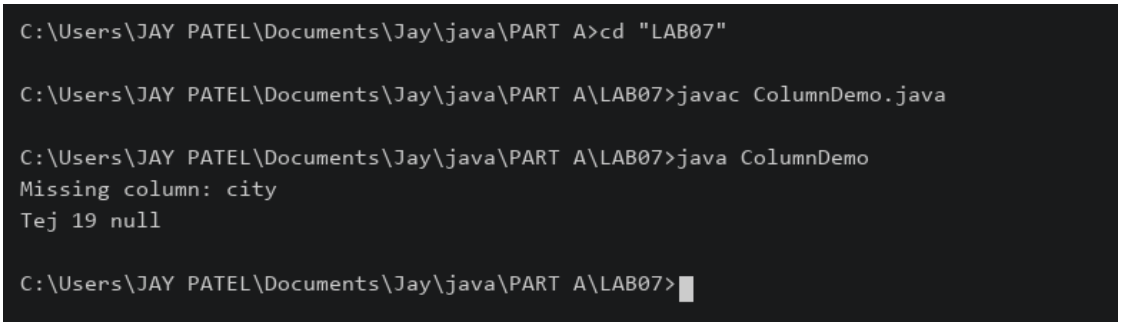
Code

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column { String name(); }
class Student {
   @Column(name="name") String name;
   @Column(name="age") int age;
   @Column(name="city") String city;
}
public class ColumnDemo {
   static void populate(Object obj, String[] headers, String[] row) throws Exception {
      for (Field f : obj.getClass().getDeclaredFields()) {
         Column column = f.getAnnotation(Column.class);
         if (column == null) { continue; }
         int index = -1;
         for (int i = 0; i < headers.length; i++) {
            if (headers[i].equals(column.name())) { index = i; break; }
         }
         if (index < 0 || index >= row.length) {
            System.out.println("Missing column: " + column.name()); continue;
         }
```

```
            f.setAccessible(true);
            if (f.getType() == int.class) { f.setInt(obj, Integer.parseInt(row[index])); }
            else { f.set(obj, row[index]); }
        }
    }
    public static void main(String[] args) throws Exception {
        Student s = new Student();
        populate(s, new String[]{"name", "age"}, new String[]{"Tej", "19"});
        System.out.println(s.name + " " + s.age + " " + s.city);
    }
}
```

A missing column is reported and the field keeps its default value.

Output



Practical 8

Part A

1) Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Calculator.java

Code

```
import java.util.Scanner;
class DivideByZeroException extends Exception {
    DivideByZeroException() { super("Cannot divide by zero"); }
}
public class Calculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean done = false;
        while (!done) {
            try {
                System.out.print("First number: ");
                double a = Double.parseDouble(sc.nextLine());
                System.out.print("Second number: ");
                double b = Double.parseDouble(sc.nextLine());
                System.out.print("Operator: ");
                String op = sc.nextLine();
                double result;
                switch (op) {
                    case "+" -> result = a + b;
                    case "-" -> result = a - b;
                    case "*" -> result = a * b;
                    case "/" -> {
                        if (b == 0) { throw new DivideByZeroException(); }
                        result = a / b;
                    }
                    default -> throw new IllegalArgumentException("Invalid operator");
                }
                System.out.println("Result: " + result); done = true;
            } catch (NumberFormatException e) { System.out.println("Invalid number"); }
            catch (DivideByZeroException e) { System.out.println(e.getMessage()); }
            catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
            finally { System.out.println("Attempt logged"); }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB08"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>javac Calculator.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>java Calculator
First number: 10
Second number: 0
Operator: /
Cannot divide by zero
Attempt logged
First number: 15
Second number: 3
Operator: /
Result: 5.0
Attempt logged

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>
```

2)	Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.
	StockDemo.java
Code	

```
import java.util.HashMap;
class OutOfStockException extends Exception {
    private int shortfall;
    OutOfStockException(int shortfall) {
        super("Short by " + shortfall); this.shortfall = shortfall;
    }
    int getShortfall() { return shortfall; }
}
class InvalidQuantityException extends Exception {
    InvalidQuantityException() { super("Quantity must be positive"); }
}
class Warehouse {
    HashMap<String,Integer> stock = new HashMap<>();
    Warehouse() { stock.put("Pen", 10); stock.put("Book", 5); }
    void issue(String item, int qty)
        throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0) { throw new InvalidQuantityException(); }
        int available = stock.getOrDefault(item, 0);
        if (qty > available) { throw new OutOfStockException(qty - available); }
        stock.put(item, available - qty);
        System.out.println("Issued " + qty + " " + item);
    }
}
public class StockDemo {
    public static void main(String[] args) {
        Warehouse w = new Warehouse();
        String[] items = {"Pen", "Book", "Pen", "Book"};
        int[] qty = {4, 8, 0, 2};
        for (int i = 0; i < items.length; i++) {
            try { w.issue(items[i], qty[i]); }
            catch (OutOfStockException e) { System.out.println(e.getMessage()); }
            catch (InvalidQuantityException e) { System.out.println(e.getMessage()); }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB08"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>javac StockDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>java StockDemo
Issued 4 Pen
Short by 3
Quantity must be positive
Issued 2 Book

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>
```

3)	(For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource.
	ResourceDemo.java
Code	

```
class ReportSession implements AutoCloseable {
    public void close() throws Exception {
        System.out.println("Session closed");
        throw new Exception("Close error");
    }
}
```

```
public class ResourceDemo {
    public static void main(String[] args) {
        try (ReportSession session = new ReportSession()) {
            System.out.println("Session opened");
            throw new Exception("Original processing error");
        } catch (Exception e) {
            System.out.println("Reported: " + e.getMessage());
            for (Throwable error : e.getSuppressed()) {
                System.out.println("Suppressed: " + error.getMessage());
            }
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB08"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>javac ResourceDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>java ResourceDemo
Session opened
Session closed
Reported: Original processing error
Suppressed: Close error

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB08>
```

Practical 9

Part A

1) Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

CounterRace.java

Code

```
public class CounterRace {
    static class Counter {
        int value;
        void unsafe() { value++; }
        synchronized void safe() { value++; }
    }
    static void run(boolean safe) throws InterruptedException {
        Counter counter = new Counter();
        Thread[] threads = new Thread[10];
        for (int i = 0; i < threads.length; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < 100000; j++) {
                    if (safe) { counter.safe(); } else { counter.unsafe(); }
                }
            });
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        System.out.println("Synchronized: " + safe + " Total: " + counter.value);
    }
    public static void main(String[] args) throws Exception { run(false); run(true); }
}
```

Race results vary; an unsafe run can occasionally produce the expected total.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB09"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>javac CounterRace.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>java CounterRace
Synchronized: false Total: 145105
Synchronized: true Total: 1000000

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>
```

2) Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.

SeatRace.java

Code

```
public class SeatRace {
    static class Booking {
        int seatsLeft = 5;
        boolean unsafe() {
            if (seatsLeft > 0) {
                try { Thread.sleep(10); }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); return false; }
                seatsLeft--; return true;
            }
            return false;
        }
        synchronized boolean book() {
            if (seatsLeft > 0) { seatsLeft--; return true; }
```

```
            return false;
        }
    }
    static void run(boolean safe) throws Exception {
        Booking booking = new Booking();
        Thread[] threads = new Thread[10];
        boolean[] success = new boolean[10];
        for (int i = 0; i < 10; i++) {
            final int id = i;
            threads[i] = new Thread(() -> {
                if (safe) { success[id] = booking.book(); }
                else { success[id] = booking.unsafe(); }
            });
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        int count = 0;
        for (boolean booked : success) { if (booked) { count++; } }
        System.out.println("Synchronized: " + safe + " Booked: " + count);
    }
    public static void main(String[] args) throws Exception { run(false); run(true); }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB09"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>javac SeatRace.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>java SeatRace
Synchronized: false Booked: 10
Synchronized: true Booked: 5

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB09>
```

3) (For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference.

SumDemo.java

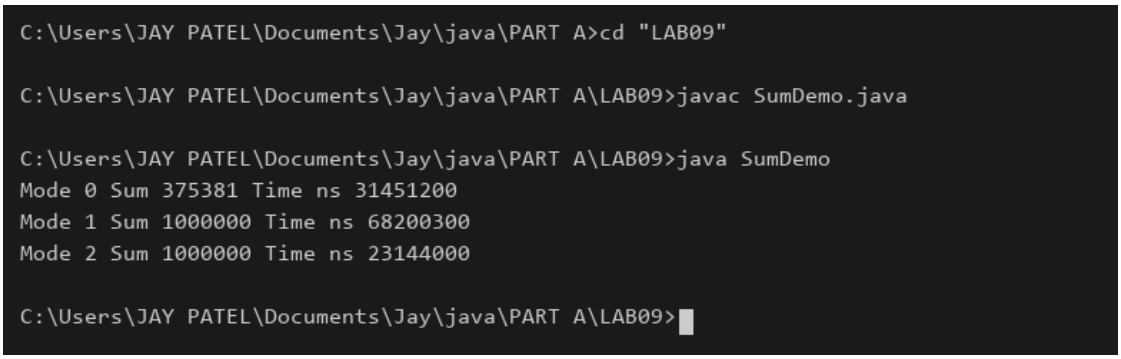
Code

```
import java.util.concurrent.atomic.AtomicLong;
public class SumDemo {
    static long total;
    static synchronized void add(long n) { total += n; }
    static void run(int mode, int[] values) throws Exception {
        total = 0;
        AtomicLong atomic = new AtomicLong();
        Thread[] threads = new Thread[4];
        long start = System.nanoTime();
        for (int i = 0; i < 4; i++) {
            int from = i * values.length / 4;
            int to = (i + 1) * values.length / 4;
            threads[i] = new Thread(() -> {
                for (int j = from; j < to; j++) {
                    if (mode == 0) { total += values[j]; }
                    else if (mode == 1) { add(values[j]); }
                    else { atomic.addAndGet(values[j]); }
                }
            });
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        long result = total;
        if (mode == 2) { result = atomic.get(); }
        System.out.println("Mode " + mode + " Sum " + result
            + " Time ns " + (System.nanoTime() - start));
    }
```

```
    public static void main(String[] args) throws Exception {
        int[] values = new int[1000000];
        java.util.Arrays.fill(values, 1);
        run(0, values); run(1, values); run(2, values);
    }
}
```

Mode 0 is unsafe; mode 1 uses synchronization; mode 2 uses AtomicLong. Times include thread creation and are illustrative.

Output



Practical 10

Part A	
1)	Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.
	PoolDemo.java

Code	`import java.util.concurrent.*;`
	`public class PoolDemo {`
	`  public static void main(String[] args) throws Exception {`
	`  ExecutorService pool = Executors.newFixedThreadPool(3);`
	`  for (int i = 1; i <= 10; i++) {`
	`  final int id = i;`
	`  pool.execute(() -> {`
	`  System.out.println("Task " + id + " " + Thread.currentThread().getName());`
	`  try { Thread.sleep(100); }`
	`  catch (InterruptedException e) { Thread.currentThread().interrupt(); }`
	`  });`
	`  }`
	`  pool.shutdown();`
	`  if (!pool.awaitTermination(1, TimeUnit.MINUTES)) { pool.shutdownNow(); }`
	`  }`
	`}`

Output	`C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB10"`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>javac PoolDemo.java`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>java PoolDemo`
	`Task 2 pool-1-thread-2`
	`Task 3 pool-1-thread-3`
	`Task 1 pool-1-thread-1`
	`Task 5 pool-1-thread-1`
	`Task 6 pool-1-thread-3`
	`Task 4 pool-1-thread-2`
	`Task 7 pool-1-thread-3`
	`Task 8 pool-1-thread-1`
	`Task 9 pool-1-thread-2`
	`Task 10 pool-1-thread-2`
	
	`C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>`

2)	Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.
	BufferDemo.java

Code	`class Buffer {`
	`  private int value;`
	`  private boolean full;`
	`  synchronized void put(int n) throws InterruptedException {`
	`  while (full) { wait(); }`
	`  value = n; full = true;`
	`  System.out.println("Produced: " + n); notify();`
	`  }`
	`  synchronized int take() throws InterruptedException {`
	`  while (!full) { wait(); }`
	`  int n = value; full = false;`
	`  System.out.println("Consumed: " + n); notify(); return n;`

```
    }
}
public class BufferDemo {
    public static void main(String[] args) throws Exception {
        Buffer buffer = new Buffer();
        Thread producer = new Thread(() -> {
            try { for (int i = 1; i <= 5; i++) { buffer.put(i); } }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        Thread consumer = new Thread(() -> {
            try { for (int i = 1; i <= 5; i++) { buffer.take(); } }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        });
        producer.start(); consumer.start(); producer.join(); consumer.join();
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB10"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>javac BufferDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>java BufferDemo
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>
```

3) (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself.

LockDemo.java

Code

```
import java.util.concurrent.CountDownLatch;
public class LockDemo {
    static void pause() {
        try { Thread.sleep(50); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
    static void lock(Object a, Object b) {
        synchronized (a) { pause(); synchronized (b) { System.out.println("Completed"); } }
    }
    public static void main(String[] args) throws Exception {
        Object a = new Object(), b = new Object();
        if (args.length > 0 && args[0].equals("deadlock")) {
            CountDownLatch held = new CountDownLatch(2);
            Runnable first = () -> {
                synchronized (a) {
                    held.countDown();
                    try { held.await(); } catch (InterruptedException e) { return; }
                    synchronized (b) { System.out.println("A then B"); }
                }
            };
            Runnable second = () -> {
                synchronized (b) {
                    held.countDown();
                    try { held.await(); } catch (InterruptedException e) { return; }
```

```
                synchronized (a) { System.out.println("B then A"); }
            }
        };
        new Thread(first).start(); new Thread(second).start();
    } else {
        Thread t1 = new Thread(() -> lock(a,b));
        Thread t2 = new Thread(() -> lock(a,b));
        t1.start(); t2.start(); t1.join(); t2.join();
    }
  }
}
```

java LockDemo deadlock deliberately hangs; stop it with Ctrl+C. java LockDemo runs the fixed version.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB10"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>javac LockDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>java LockDemo
Completed
Completed

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB10>
```

Practical 11

Part A	
1)	Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.
	SaveDemo.java
Code	

```
import java.io.*;
class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    String name;
    int marks;
    transient String password;
    Student(String name, int marks, String password) {
        this.name = name; this.marks = marks; this.password = password;
    }
}
public class SaveDemo {
    public static void main(String[] args) throws Exception {
        Student[] students = {new Student("Tej",90,"abc"), new Student("Riya",85,"xyz")};
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("students.dat")))
{
            out.writeObject(students);
        }
        Student[] loaded;
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("students.dat"))) {
            loaded = (Student[]) in.readObject();
        }
        for (Student s : loaded) {
            System.out.println(s.name + " " + s.marks + " Password: " + s.password);
        }
        System.out.println("Fresh object: " + (students[0] != loaded[0]));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB11"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>javac SaveDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>java SaveDemo
Tej 90 Password: null
Riya 85 Password: null
Fresh object: true

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>
```

2)	Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.
	LogAnalyzer.java
Code	

```
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.io.BufferedReader;
public class LogAnalyzer {
    public static void main(String[] args) throws Exception {
        Path folder = Path.of("samplelogs"); Files.createDirectories(folder);
        Files.writeString(folder.resolve("one.log"), "login OK\nlogin ERROR\n");
        Files.writeString(folder.resolve("two.log"), "payment ERROR\n");
        String keyword = "ERROR";
```

```
        int lines = 0, matches = 0;
        try (DirectoryStream<Path> files = Files.newDirectoryStream(folder, "*.log")) {
            for (Path file : files) {
                BasicFileAttributes attr = Files.readAttributes(file, BasicFileAttributes.class);
                System.out.println(file + " Size: " + attr.size());
                try (BufferedReader reader = Files.newBufferedReader(file)) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        lines++;
                        if (line.contains(keyword)) { matches++; }
                    }
                }
            }
        }
        System.out.println("Lines: " + lines + " Matches: " + matches);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB11"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>javac LogAnalyzer.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>java LogAnalyzer
samplelogs\one.log Size: 21
samplelogs\two.log Size: 14
Lines: 3 Matches: 2

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>
```

3) (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

TreeReport.java

Code

```
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.stream.Stream;
public class TreeReport {
    public static void main(String[] args) throws Exception {
        Path root = Path.of("sampletree"); Files.createDirectories(root.resolve("sub"));
        Files.writeString(root.resolve("sub/a.txt"), "Hello");
        Path report = Path.of("tree-report.txt");
        StringBuilder text = new StringBuilder();
        try (Stream<Path> paths = Files.walk(root)) {
            for (Path file : paths.filter(Files::isRegularFile).toList()) {
                BasicFileAttributes a = Files.readAttributes(file, BasicFileAttributes.class);
                text.append(root.relativize(file)).append(" ").append(a.size())
                    .append(" ").append(a.lastModifiedTime()).append("\n");
            }
        }
        Files.writeString(report, text); System.out.print(text);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB11"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>javac TreeReport.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>java TreeReport
sub\a.txt 5 2026-10-28T15:56:20.4590231Z

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB11>
```

Practical 12

Part A

1) Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

WordFrequency.java

Code

```
import java.util.*;
public class WordFrequency {
    public static void main(String[] args) {
        String paragraph = "Java is simple. Java is useful. Java supports objects.";
        HashMap<String,Integer> counts = new HashMap<>();
        for (String word : paragraph.toLowerCase().split("[^a-z]+")) {
            if (!word.isEmpty()) { counts.put(word, counts.getOrDefault(word,0) + 1); }
        }
        ArrayList<Map.Entry<String,Integer>> list = new ArrayList<>(counts.entrySet());
        list.sort(Comparator.<Map.Entry<String,Integer>>comparingInt(e -> e.getValue())
            .reversed().thenComparing(e -> e.getKey()));
        for (Map.Entry<String,Integer> e : list) {
            System.out.println(e.getKey() + ": " + e.getValue());
        }
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB12"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB12>javac WordFrequency.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB12>java WordFrequency
java: 3
is: 2
objects: 1
simple: 1
supports: 1
useful: 1

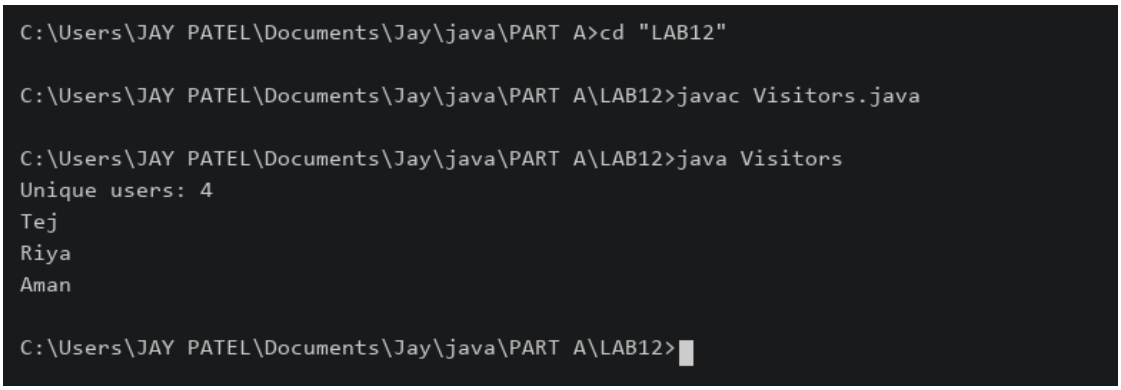
C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB12>
```

2) Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

Visitors.java

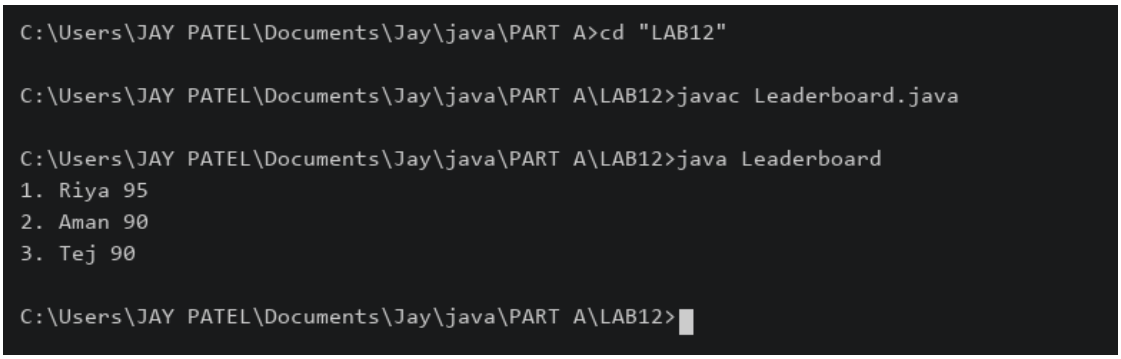
Code

```
import java.util.*;
public class Visitors {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Tej","Riya","Tej","Aman","Riya","Neha");
        HashSet<String> unique = new HashSet<>(names);
        LinkedHashSet<String> ordered = new LinkedHashSet<>(names);
        System.out.println("Unique users: " + unique.size());
        int count = 0;
        for (String name : ordered) {
            if (count == 3) { break; }
            System.out.println(name); count++;
        }
    }
}
```

Output	

3)	(For Advanced Learners – home) Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details.
	Leaderboard.java

Code	import java.util.*;
	public class Leaderboard {
	public static void main(String[] args) {
	Map<String,Integer> scores = new HashMap<>();
	scores.put("Tej",90); scores.put("Riya",95);
	scores.put("Aman",90); scores.put("Neha",80);
	ArrayList<Map.Entry<String,Integer>> list = new ArrayList<>(scores.entrySet());
	list.sort(Comparator.<Map.Entry<String,Integer>>comparingInt(e -> e.getValue())
	.reversed().thenComparing(e -> e.getKey()));
	int n = 3;
	for (int i = 0; i < Math.min(n,list.size()); i++) {
	System.out.println((i + 1) + ". " + list.get(i).getKey()
	+ " " + list.get(i).getValue());
	}
	}
	}

Output	

Practical 13

Part A

1) Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

ContainerDemo.java

Code

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
class Pair<A,B> {
    private A first; private B second;
    Pair(A first, B second) { this.first = first; this.second = second; }
    A getFirst() { return first; }
    B getSecond() { return second; }
}
public class ContainerDemo {
    static <T> T first(T[] values) {
        if (values.length == 0) { throw new IllegalArgumentException("Empty array"); }
        return values[0];
    }
    public static void main(String[] args) {
        Box<String> names = new Box<>(); names.set("Tej");
        Box<Integer> marks = new Box<>(); marks.set(90);
        Pair<String,Integer> pair = new Pair<>(names.get(), marks.get());
        System.out.println(pair.getFirst() + " " + pair.getSecond());
        System.out.println(first(new String[]{"Java","Python"}));
        System.out.println(first(new Integer[]{10,20}));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB13"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>javac ContainerDemo.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>java ContainerDemo
Tej 90
Java
10

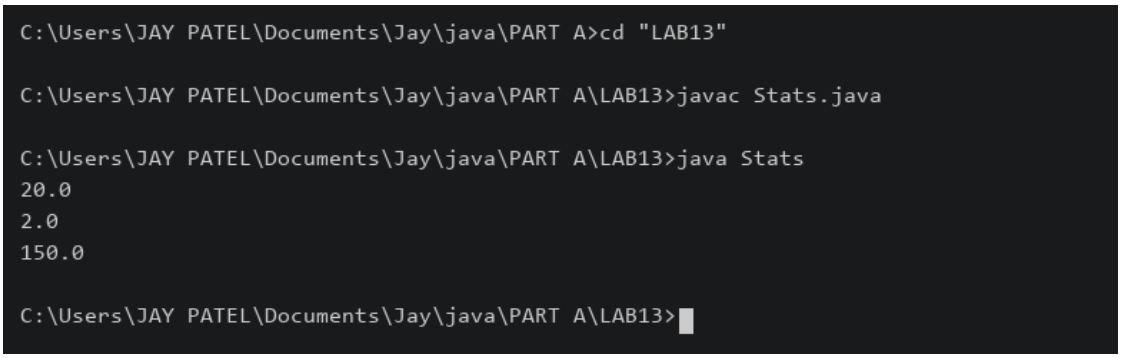
C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB13>
```

2) Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

Stats.java

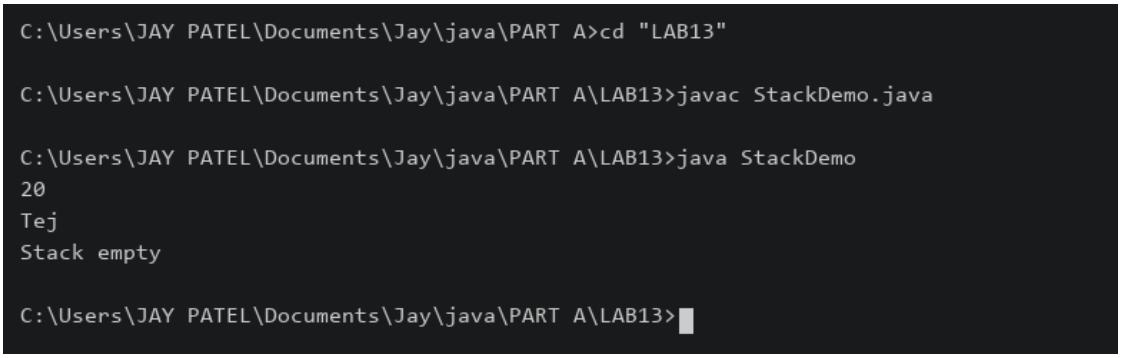
Code

```
public class Stats {
    static <N extends Number> double average(N[] values) {
        if (values.length == 0) { throw new IllegalArgumentException("Empty array"); }
        double sum = 0;
        for (N n : values) { sum += n.doubleValue(); }
        return sum / values.length;
    }
    public static void main(String[] args) {
        System.out.println(average(new Integer[]{10,20,30}));
        System.out.println(average(new Double[]{1.5,2.5}));
        System.out.println(average(new Long[]{100L,200L}));
    }
```

	}
	String[] is rejected at compile time because String does not extend Number.
Output	

3)	(For Advanced Learners – home) Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.
	StackDemo.java

Code	import java.util.EmptyStackException;
	class Stack<T> {
	private Object[] data;
	private int size;
	Stack(int capacity) { data = new Object[capacity]; }
	void push(T item) {
	if (size == data.length) { throw new IllegalStateException("Stack full"); }
	data[size++] = item;
	}
	@SuppressWarnings("unchecked")
	T pop() {
	if (size == 0) { throw new EmptyStackException(); }
	T item = (T) data[--size]; data[size] = null; return item;
	}
	}
	public class StackDemo {
	public static void main(String[] args) {
	Stack<Integer> numbers = new Stack<>(3); numbers.push(10); numbers.push(20);
	Stack<String> names = new Stack<>(2); names.push("Tej");
	System.out.println(numbers.pop()); System.out.println(names.pop());
	try { names.pop(); }
	catch (EmptyStackException e) { System.out.println("Stack empty"); }
	}
	}

Output	

Practical 14

Part A	
1)	Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.
	Item.java
Code	public record Item(String name) {}
	ItemService.java

Code	import java.util.ArrayList;
	public class ItemService {
	private ArrayList<Item> items = new ArrayList<>();
	public void add(String name) {
	if (name == null || name.isBlank()) { throw new IllegalArgumentException("Name required"); }
	items.add(new Item(name));
	}
	public int count() { return items.size(); }
	public ArrayList<Item> all() { return new ArrayList<>(items); }
	}

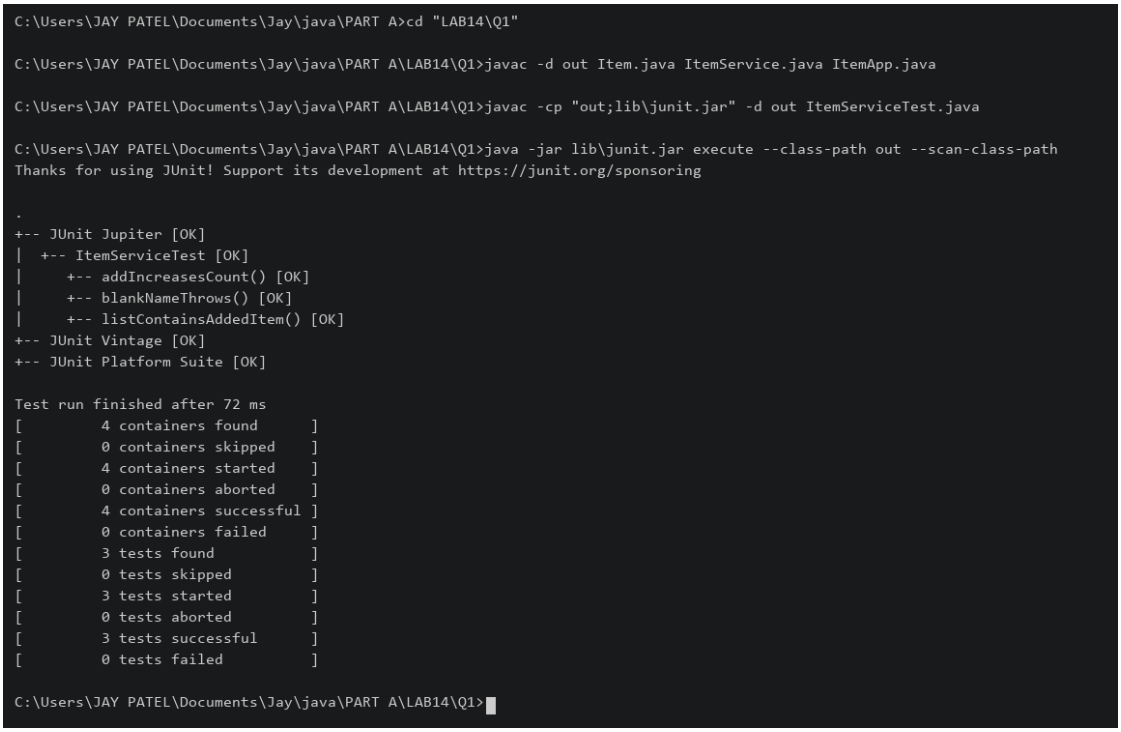
	ItemApp.java

Code	import java.util.Scanner;
	public class ItemApp {
	public static void main(String[] args) {
	ItemService service = new ItemService();
	Scanner sc = new Scanner(System.in);
	while (true) {
	System.out.println("1. Add\n2. List\n3. Exit");
	String choice = sc.nextLine();
	if (choice.equals("3")) { break; }
	try {
	if (choice.equals("1")) {
	System.out.print("Name: "); service.add(sc.nextLine());
	} else if (choice.equals("2")) { System.out.println(service.all()); }
	else { System.out.println("Invalid choice"); }
	} catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
	}
	}
	}

	ItemServiceTest.java

Code	import org.junit.jupiter.api.Test;
	import static org.junit.jupiter.api.Assertions.*;
	public class ItemServiceTest {
	@Test void addIncreasesCount() {
	ItemService service = new ItemService();
	service.add("Book"); assertEquals(1,service.count());
	}
	@Test void blankNameThrows() {
	assertThrows(IllegalArgumentException.class, () -> new ItemService().add(" "));
	}
	@Test void listContainsAddedItem() {
	ItemService service = new ItemService(); service.add("Pen");
	assertEquals("Pen",service.all().get(0).name());
	}
	}

	Place JUnit Platform Console Standalone 1.10.2 in lib/junit.jar. javac -d out Item.java ItemService.java ItemApp.java javac -cp "out;lib/junit.jar" -d out ItemServiceTest.java

	java -jar lib/junit.jar execute --class-path out --scan-class-path On Linux/macOS replace the classpath semicolon with a colon.
Output	

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB14\Q1"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>javac -d out Item.java ItemService.java ItemApp.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>javac -cp "out;lib\junit.jar" -d out ItemServiceTest.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>java -jar lib\junit.jar execute --class-path out --scan-class-path
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Jupiter [OK]
|  +-- ItemServiceTest [OK]
|     +-- addIncreasesCount() [OK]
|     +-- blankNameThrows() [OK]
|     +-- listContainsAddedItem() [OK]
+-- JUnit Vintage [OK]
+-- JUnit Platform Suite [OK]

Test run finished after 72 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         3 tests found           ]
[         0 tests skipped         ]
[         3 tests started         ]
[         0 tests aborted         ]
[         3 tests successful      ]
[         0 tests failed          ]

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q1>
```

2)	(For Advanced Learners – home) Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged.
	model/Item.java
Code	package model; import java.io.Serializable; public record Item(String name) implements Serializable {}
	service/ItemService.java

Code	package service;
	import model.Item;
	import java.io.*;
	import java.nio.file.*;
	import java.util.ArrayList;
	public class ItemService {
	  private ArrayList<Item> items = new ArrayList<>();
	  public void add(String name) {
	  if (name == null || name.isBlank()) { throw new IllegalArgumentException("Name required"); }
	  items.add(new Item(name));
	  }
	  public void save(Path file) throws IOException {
	  try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
	  out.writeObject(items);
	  }
	  }
	  @SuppressWarnings("unchecked")
	  public void load(Path file) throws IOException, ClassNotFoundException {
	  try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
	  items = (ArrayList<Item>) in.readObject();
	  }
	  }
	  public int count() { return items.size(); }
	  public ArrayList<Item> all() { return new ArrayList<>(items); }
	}

	app/ItemApp.java

Code	package app;
	import service.ItemService;
	import java.util.Scanner;
	public class ItemApp {
	public static void main(String[] args) {
	ItemService service = new ItemService();
	Scanner sc = new Scanner(System.in);
	while (true) {
	System.out.println("1. Add\n2. List\n3. Exit");
	String choice = sc.nextLine();
	if (choice.equals("3")) { break; }
	try {
	if (choice.equals("1")) {
	System.out.print("Name: "); service.add(sc.nextLine());
	} else if (choice.equals("2")) { System.out.println(service.all()); }
	else { System.out.println("Invalid choice"); }
	} catch (IllegalArgumentException e) { System.out.println(e.getMessage()); }
	}
	}
	}

	test/ItemServiceTest.java

Code	package test;
	import service.ItemService;
	import java.nio.file.*;
	import org.junit.jupiter.api.Test;
	import static org.junit.jupiter.api.Assertions.*;
	public class ItemServiceTest {
	@Test void addIncreasesCount() {
	ItemService service = new ItemService();
	service.add("Book"); assertEquals(1,service.count());
	}
	@Test void blankNameThrows() {
	assertThrows(IllegalArgumentException.class, () -> new ItemService().add(" "));
	}
	@Test void listContainsAddedItem() {
	ItemService service = new ItemService(); service.add("Pen");
	assertEquals("Pen",service.all().get(0).name());
	}
	@Test void saveAndLoad() throws Exception {
	Path file = Files.createTempFile("items", ".dat");
	try {
	ItemService first = new ItemService(); first.add("Pen"); first.save(file);
	ItemService second = new ItemService(); second.load(file);
	assertEquals(first.all(),second.all());
	} finally { Files.deleteIfExists(file); }
	}
	}

	Build from the folder containing model, service, app and test. javac -d out model/Item.java service/ItemService.java app/ItemApp.java jar --create --file items.jar --main-class app.ItemApp -C out . java -jar items.jar javac -cp "out;lib/junit.jar" -d test-out test/ItemServiceTest.java java -jar lib/junit.jar execute --class-path "out;test-out" --scan-class-path On Linux/macOS use a colon in classpaths.

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART A>cd "LAB14\Q2"

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>javac -d out model\Item.java service\ItemService.java app\ItemApp.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>jar --create --file items.jar --main-class app.ItemApp -C out .

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>javac -cp "out;lib\junit.jar" -d test-out test\ItemServiceTest.java

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>java -jar lib\junit.jar execute --class-path "out;test-out" --scan-class-path
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Jupiter [OK]
|  +-- ItemServiceTest [OK]
|     +-- addIncreasesCount() [OK]
|     +-- blankNameThrows() [OK]
|     +-- saveAndLoad() [OK]
|     +-- listContainsAddedItem() [OK]
+-- JUnit Vintage [OK]
+-- JUnit Platform Suite [OK]

Test run finished after 183 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         4 tests found           ]
[         0 tests skipped         ]
[         4 tests started         ]
[         0 tests aborted         ]
[         4 tests successful      ]
[         0 tests failed          ]

C:\Users\JAY PATEL\Documents\Jay\java\PART A\LAB14\Q2>
```

Practical 1

Part B	
1)	1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>; add the faculty as a collaborator. 2. Create a public class named MiniBank that contains the main method. 3. Define a record named BankInfo with two String fields, name and branch (a record is a short, read-only class; Java auto-generates its constructor and field accessors). Print one BankInfo object as the application header. 4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer. 5. Display the numbered menu to the user and read the chosen number with a Scanner. 6. Use a switch expression on the choice to print a short placeholder for that option (for example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a loop until the user selects EXIT, then print a goodbye message.
	Create a private repository oop-minibank-25AIML057 and add the faculty collaborator using the faculty GitHub username. Install JDK 17 or newer. Save this stage in lab-01. Compile: javac MiniBank.java Run: java MiniBank Commit and tag: git add . then git commit -m "Practical 1" then git tag lab-01. README: Compile with javac MiniBank.java and run with java MiniBank. Select 1 to 5; option 5 exits.
	MiniBank.java

Code	import java.util.Scanner;
	record BankInfo(String name, String branch) {}
	enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT }
	public class MiniBank {
	public static void main(String[] args) {
	System.out.println(new BankInfo("MiniBank", "Anand"));
	Scanner sc = new Scanner(System.in);
	while (true) {
	System.out.println("1. Open account\n2. Deposit\n3. Withdraw\n4. Transfer\n5. Exit");
	int choice;
	try { choice = Integer.parseInt(sc.nextLine()); }
	catch (NumberFormatException e) { System.out.println("Invalid choice"); continue; }
	if (choice < 1 || choice > 5) { System.out.println("Invalid choice"); continue; }
	MenuOption option = MenuOption.values()[choice - 1];
	String message = switch (option) {
	case OPEN_ACCOUNT -> "Open account - to be implemented later";
	case DEPOSIT -> "Deposit - to be implemented later";
	case WITHDRAW -> "Withdraw - to be implemented later";
	case TRANSFER -> "Transfer - to be implemented later";
	case EXIT -> "Goodbye";
	};
	System.out.println(message);
	if (option == MenuOption.EXIT) { break; }
	}
	}
	}

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB01>javac MiniBank.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB01>java MiniBank
BankInfo[name=MiniBank, branch=Anand]
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
1
Open account - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
2
Deposit - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
3
Withdraw - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
4
Transfer - to be implemented later
1. Open account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
5
Goodbye

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB01>
```

Practical 2

Part B	
1)	1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as “CUST101”. Add a constructor and public getter methods. 2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example “AC0001”). 3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0). 4. Add a public method deposit(long amount) that adds amount to balance. Add a public method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods. Do NOT add a public setter for balance. 5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.
	Continue with Practical 1. Add these files; keep MiniBank.java. javac *.java java Lab02 Commit and tag lab-02.
	Customer.java

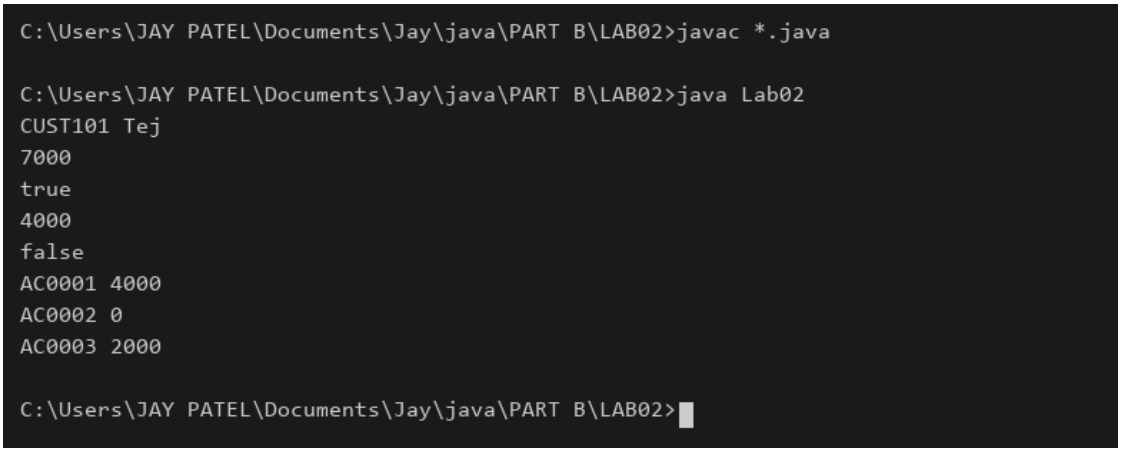
Code	public class Customer {
	private String name, email, mobile;
	private final String customerId;
	private static long customerCounter = 100;
	private static String generateCustomerId() { return "CUST" + (++customerCounter); }
	public Customer(String name, String email, String mobile) {
	this.name = name; this.email = email; this.mobile = mobile;
	customerId = generateCustomerId();
	}
	public String getName() { return name; }
	public String getEmail() { return email; }
	public String getMobile() { return mobile; }
	public String getCustomerId() { return customerId; }
	}

	Account.java

Code	public class Account {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	public Account(String ownerName) { this(ownerName, 0); }
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && amount <= balance) { balance -= amount; return true; }
	return false;
	}
	public String getAccountNumber() { return accountNumber; }

	public String getOwnerName() { return ownerName; } public long getBalance() { return balance; } public boolean isActive() { return active; } }
	Lab02.java

Code	public class Lab02 {
	public static void main(String[] args) {
	Customer customer = new Customer("Tej", "tej@test.com", "9876543210");
	System.out.println(customer.getCustomerId() + " " + customer.getName());
	Account[] accounts = {new Account("Riya",5000),new Account("Tej"),new Account("Aman",2000)};
	accounts[0].deposit(2000); System.out.println(accounts[0].getBalance());
	System.out.println(accounts[0].withdraw(3000));
	System.out.println(accounts[0].getBalance());
	System.out.println(accounts[0].withdraw(10000));
	for (Account a : accounts) {
	System.out.println(a.getAccountNumber() + " " + a.getBalance());
	}
	}
	}

Output	

Practical 3

Part B	
1)	1. In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance. 2. In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal. 3. In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method. 4. In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer. 5. In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.
	Replace Account.java and Customer.java from Practical 2. Add Lab03.java. The Account copy constructor is for the equality demonstration; do not register copies as new accounts. Address is immutable, so sharing it in clone is safe. javac *.java java Lab03 Commit and tag lab-03.
	Account.java

Code	public class Account {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	public Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && amount <= balance) { balance -= amount; return true; }
	return false;
	}
	public String getAccountNumber() { return accountNumber; }
	public String getOwnerName() { return ownerName; }
	public long getBalance() { return balance; }
	public boolean isActive() { return active; }
	}

	Customer.java
Code	public class Customer implements Cloneable { public static class Address {

```
        private final String line, city, pincode;
        public Address(String line, String city, String pincode) {
            this.line = line; this.city = city; this.pincode = pincode;
        }
        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }
    }
    private Address address;
    public Customer(String name, String email, String mobile, Address address) {
        this(name,email,mobile); this.address = address;
    }
    public Address getAddress() { return address; }
    public Customer clone() {
        try { return (Customer) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }
    }
    private String name, email, mobile;
    private final String customerId;
    private static long customerCounter = 100;
    private static String generateCustomerId() { return "CUST" + (++customerCounter); }
    public Customer(String name, String email, String mobile) {
        this.name = name; this.email = email; this.mobile = mobile;
        customerId = generateCustomerId();
    }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
    public String getCustomerId() { return customerId; }
}
```

Lab03.java

Code

```
public class Lab03 {
    public static void main(String[] args) {
        Account first = new Account("Riya",4000);
        Account copy = new Account(first);
        System.out.println(first); System.out.println(copy);
        System.out.println(first.equals(copy));
        Object obj = first; System.out.println(obj instanceof Account);
        Customer c = new Customer("Tej","tej@test.com","9876543210",
            new Customer.Address("Main Road","Anand","388001"));
        Customer clone = c.clone();
        System.out.println(clone.getName() + " " + clone.getAddress().getCity());
        System.out.println("Separate customer object: " + (c != clone));
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB03>javac *.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB03>java Lab03
AC0001 | Riya | 4000 | true
AC0001 | Riya | 4000 | true
true
true
Tej Anand
Separate customer object: true

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB03>
```

Practical 4

Part B	
1)	1. Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean. 2. Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount. 3. Create a class named CommandParser with a public static method parse(String line) that splits a line such as “DEPOSIT AC0001 500” with split(), converts the parts, and returns a Command object. 4. Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string. 5. In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.
	Add these files to the Practical 3 project. javac *.java java Lab04 Commit and tag lab-04.
	Validator.java

Code	import java.util.regex.Pattern;
	public class Validator {
	private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
	private static final Pattern EMAIL = Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
	private static final Pattern PAN = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
	private static final Pattern IFSC = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");
	private static boolean matches(Pattern p, String value) {
	return value != null && p.matcher(value).matches();
	}
	public static boolean isValidMobile(String s) { return matches(MOBILE,s); }
	public static boolean isValidEmail(String s) { return matches(EMAIL,s); }
	public static boolean isValidPan(String s) { return matches(PAN,s); }
	public static boolean isValidIfsc(String s) { return matches(IFSC,s); }
	}

	TransactionType.java
Code	public enum TransactionType { DEPOSIT, WITHDRAW, TRANSFER }
	Command.java
Code	public record Command(TransactionType type, String accountNumber, long amount) {}
	CommandParser.java

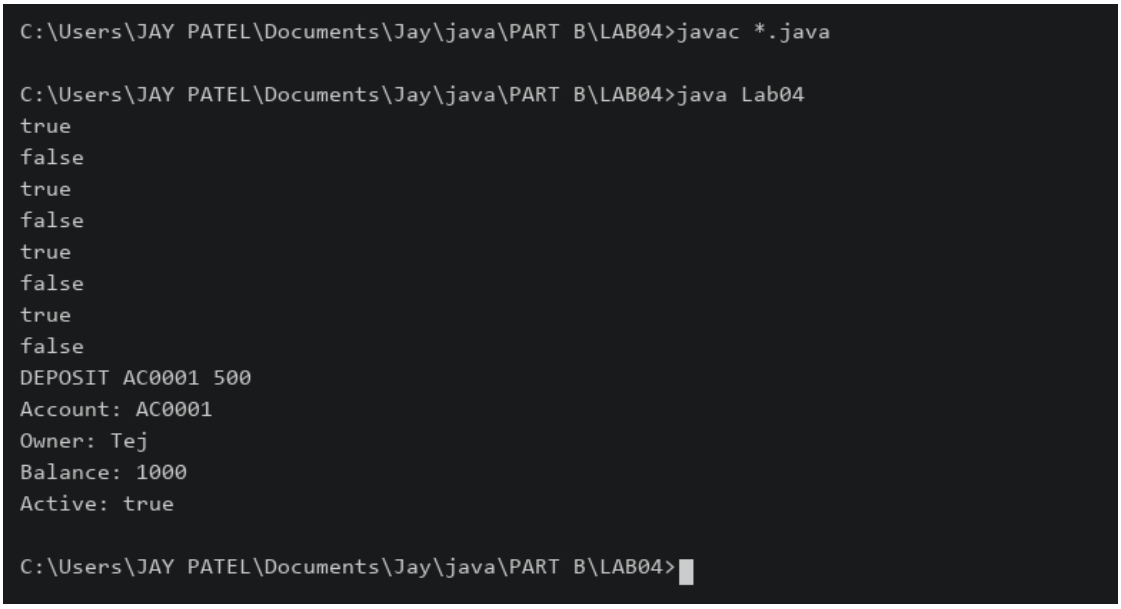
Code	public class CommandParser {
	public static Command parse(String line) {
	String[] parts = line.trim().split("\\s+");
	if (parts.length != 3) { throw new IllegalArgumentException("Use TYPE ACCOUNT AMOUNT"); }
	long amount = Long.parseLong(parts[2]);
	if (amount <= 0) { throw new IllegalArgumentException("Amount must be positive"); }
	return new Command(TransactionType.valueOf(parts[0].toUpperCase()),parts[1],amount);
	}
	}

	StatementFormatter.java

Code	public class StatementFormatter {
	  public static String buildStatement(Account a) {
	  return new StringBuilder().append("Account: ").append(a.getAccountNumber())
	  .append("\nOwner: ").append(a.getOwnerName()).append("\nBalance: ")
	  .append(a.getBalance()).append("\nActive: ").append(a.isActive()).toString();
	  }
	}

	Lab04.java

Code	public class Lab04 {
	  public static void main(String[] args) {
	  System.out.println(Validator.isValidMobile("9876543210"));
	  System.out.println(Validator.isValidMobile("12345"));
	  System.out.println(Validator.isValidEmail("abc@xyz.com"));
	  System.out.println(Validator.isValidEmail("abc"));
	  System.out.println(Validator.isValidPan("ABCDE1234F"));
	  System.out.println(Validator.isValidPan("1234"));
	  System.out.println(Validator.isValidIfsc("SBIN0001234"));
	  System.out.println(Validator.isValidIfsc("INVALID"));
	  Command c = CommandParser.parse("DEPOSIT AC0001 500");
	  System.out.println(c.type() + " " + c.accountNumber() + " " + c.amount());
	  System.out.println(StatementFormatter.buildStatement(new Account("Tej",1000)));
	  }
	}

Output	

Practical 5

Part B	
1)	1. Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount). 2. Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance. 3. Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit. 4. Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked). 5. Call super(...) from each subclass constructor to initialise the inherited fields. 6. In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially.
	Replace Account.java and add the subclasses and Lab05.java. Keep earlier driver files in their earlier lab folders; they instantiate the now abstract Account. Compile only the current model, utility files and driver. javac Account.java Customer.java SavingsAccount.java CurrentAccount.java FixedDepositAccount.java Validator.java TransactionType.java Command.java CommandParser.java StatementFormatter.java Lab05.java java Lab05 Commit and tag lab-05.
	Account.java

Code	public abstract class Account {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && canWithdraw(amount)) { balance -= amount; return true; }
	return false;
	}

```
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

SavingsAccount.java

Code

```
public class SavingsAccount extends Account {
    private long minBalance;
    public SavingsAccount(String name, long balance, long minBalance) {
        super(name,balance); this.minBalance = minBalance;
    }
    public double interestRate() { return 4.0; }
    public boolean canWithdraw(long amount) {
        return amount > 0 && amount <= getBalance() - minBalance;
    }
}
```

CurrentAccount.java

Code

```
public class CurrentAccount extends Account {
    private long overdraftLimit;
    public CurrentAccount(String name, long balance, long overdraftLimit) {
        super(name,balance); this.overdraftLimit = overdraftLimit;
    }
    public double interestRate() { return 0; }
    public boolean canWithdraw(long amount) {
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }
}
```

FixedDepositAccount.java

Code

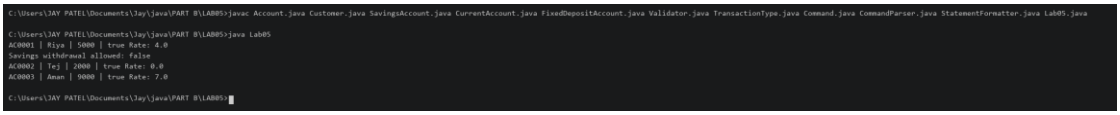
```
public class FixedDepositAccount extends Account {
    public FixedDepositAccount(String name, long balance) { super(name,balance); }
    public double interestRate() { return 7.0; }
    public boolean canWithdraw(long amount) { return false; }
}
```

Lab05.java

Code

```
public class Lab05 {
    public static void main(String[] args) {
        Account[] accounts = {new SavingsAccount("Riya",5000,1000),
            new CurrentAccount("Tej",2000,1000),new FixedDepositAccount("Aman",9000)};
        for (Account a : accounts) {
            System.out.println(a + " Rate: " + a.interestRate());
            if (a instanceof SavingsAccount s) {
                System.out.println("Savings withdrawal allowed: " + s.canWithdraw(4500));
            }
        }
    }
}
```

Output



Practical 6

Part B	
1)	1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it. 2. Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance. 3. Create a functional interface named WithdrawRule with a single method boolean allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression. 4. Create a marker interface named Premium (an interface with no methods). 5. Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import. 6. Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar.
	Move the current project into these packages; use the complete files below instead of the earlier default-package files. Keep old lab drivers outside this source tree. javac -d out model/*.java util/*.java service/*.java java -cp out service.Lab06 jar cfm minibank.jar manifest.mf -C out . java -jar minibank.jar Keep the blank line at the end of manifest.mf. Commit and tag lab-06.
	model/Account.java

Code	package model;
	public abstract class Account implements Transactable, InterestBearing {
	private final String accountNumber;
	private String ownerName;
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	public void deposit(long amount) {
	if (amount > 0) { balance += amount; }
	}
	public boolean withdraw(long amount) {
	if (amount > 0 && canWithdraw(amount)) { balance -= amount; return true; }
	return false;
	}
	public String getAccountNumber() { return accountNumber; }
	public String getOwnerName() { return ownerName; }
	public long getBalance() { return balance; }

	public boolean isActive() { return active; } }
	model/Customer.java

Code	package model;
	public class Customer implements Cloneable {
	public static class Address {
	private final String line, city, pincode;
	public Address(String line, String city, String pincode) {
	this.line = line; this.city = city; this.pincode = pincode;
	}
	public String getLine() { return line; }
	public String getCity() { return city; }
	public String getPincode() { return pincode; }
	}
	private Address address;
	public Customer(String name, String email, String mobile, Address address) {
	this(name,email,mobile); this.address = address;
	}
	public Address getAddress() { return address; }
	public Customer clone() {
	try { return (Customer) super.clone(); }
	catch (CloneNotSupportedException e) { throw new AssertionError(e); }
	}
	private String name, email, mobile;
	private final String customerId;
	private static long customerCounter = 100;
	private static String generateCustomerId() { return "CUST" + (++customerCounter); }
	public Customer(String name, String email, String mobile) {
	this.name = name; this.email = email; this.mobile = mobile;
	customerId = generateCustomerId();
	}
	public String getName() { return name; }
	public String getEmail() { return email; }
	public String getMobile() { return mobile; }
	public String getCustomerId() { return customerId; }
	}

	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	public double interestRate() { return 4.0; }
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	public double interestRate() { return 0; }
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	model/FixedDepositAccount.java
Code	package model; public class FixedDepositAccount extends Account {

	public FixedDepositAccount(String name, long balance) { super(name,balance); } public double interestRate() { return 7.0; } public boolean canWithdraw(long amount) { return false; } }
	model/TransactionType.java
Code	package model; public enum TransactionType { DEPOSIT, WITHDRAW, TRANSFER }
	model/Command.java
Code	package model; public record Command(TransactionType type, String accountNumber, long amount) {}
	model/Transactable.java

Code	package model;
	public interface Transactable {
	void deposit(long amount);
	boolean withdraw(long amount);
	}

	model/InterestBearing.java

Code	package model;
	public interface InterestBearing {
	double interestRate();
	long getBalance();
	default double yearlyInterest() { return getBalance() * interestRate() / 100; }
	}

	model/Premium.java
Code	package model; public interface Premium {}
	util/Validator.java

Code	package util;
	import java.util.regex.Pattern;
	public class Validator {
	private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
	private static final Pattern EMAIL = Pattern.compile("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
	private static final Pattern PAN = Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");
	private static final Pattern IFSC = Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");
	private static boolean matches(Pattern p, String value) {
	return value != null && p.matcher(value).matches();
	}
	public static boolean isValidMobile(String s) { return matches(MOBILE,s); }
	public static boolean isValidEmail(String s) { return matches(EMAIL,s); }
	public static boolean isValidPan(String s) { return matches(PAN,s); }
	public static boolean isValidIfsc(String s) { return matches(IFSC,s); }
	}

	util/CommandParser.java

Code	package util;
	import model.*;
	
	public class CommandParser {
	public static Command parse(String line) {
	String[] parts = line.trim().split("\\s+");
	if (parts.length != 3) { throw new IllegalArgumentException("Use TYPE ACCOUNT AMOUNT"); }
	long amount = Long.parseLong(parts[2]);
	if (amount <= 0) { throw new IllegalArgumentException("Amount must be positive"); }
	return new Command(TransactionType.valueOf(parts[0].toUpperCase()),parts[1],amount);
	}
	}

	util/StatementFormatter.java

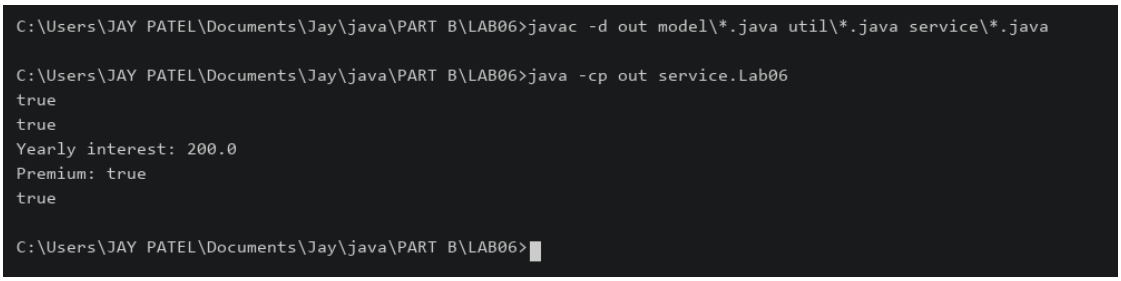
Code	package util;
	import model.Account;
	
	public class StatementFormatter {
	  public static String buildStatement(Account a) {
	  return new StringBuilder().append("Account: ").append(a.getAccountNumber())
	  .append("\nOwner: ").append(a.getOwnerName()).append("\nBalance: ")
	  .append(a.getBalance()).append("\nActive: ").append(a.isActive()).toString();
	  }
	}

	service/WithdrawRule.java
Code	package service; import model.Account; public interface WithdrawRule { boolean allow(Account account, long amount); }
	service/Lab06.java

Code	package service;
	import model.*;
	import static util.Validator.isValidMobile;
	public class Lab06 {
	  public static void main(String[] args) {
	  Account a = new SavingsAccount("Tej",5000,1000);
	  WithdrawRule first = new WithdrawRule() {
	  public boolean allow(Account a, long amount) { return a.canWithdraw(amount); }
	  };
	  WithdrawRule second = (account, amount) -> account.canWithdraw(amount);
	  System.out.println(first.allow(a,500));
	  System.out.println(second.allow(a,500));
	  System.out.println("Yearly interest: " + a.yearlyInterest());
	  System.out.println("Premium: " + (a instanceof Premium));
	  System.out.println(isValidMobile("9876543210"));
	  }
	}

	service/MiniBank.java

Code	package service;
	import java.util.Scanner;
	record BankInfo(String name, String branch) {}
	enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER, EXIT }
	public class MiniBank {
	  public static void main(String[] args) {
	  System.out.println(new BankInfo("MiniBank", "Anand"));
	  Scanner sc = new Scanner(System.in);
	  while (true) {
	  System.out.println("1. Open account\n2. Deposit\n3. Withdraw\n4. Transfer\n5. Exit");
	  int choice;
	  try { choice = Integer.parseInt(sc.nextLine()); }
	  catch (NumberFormatException e) { System.out.println("Invalid choice"); continue; }
	  if (choice < 1 || choice > 5) { System.out.println("Invalid choice"); continue; }
	  MenuOption option = MenuOption.values()[choice - 1];
	  String message = switch (option) {
	  case OPEN_ACCOUNT -> "Open account - to be implemented later";
	  case DEPOSIT -> "Deposit - to be implemented later";
	  case WITHDRAW -> "Withdraw - to be implemented later";
	  case TRANSFER -> "Transfer - to be implemented later";
	  case EXIT -> "Goodbye";
	  };
	  System.out.println(message);
	  if (option == MenuOption.EXIT) { break; }
	  }
	  }
	}

	manifest.mf
Text	Manifest-Version: 1.0 Main-Class: service.MiniBank
Output	

Practical 7

Part B	
1)	1. Add the @Override annotation above every overridden method written so far (toString, equals, hashCode and the subclass methods). Add @FunctionalInterface above the WithdrawRule interface. 2. In a package named model.annotation, define three custom annotations: @Id (a marker, with @Target(FIELD) and @Retention(RUNTIME)); @Positive with a member String message() default “must be > 0”; and @MaxLength with a member int value(). 3. Put @Id on the accountNumber field and @Positive on the balance field of Account. 4. Create a class named AnnotationValidator with a public static method String[] validate(Object obj) that uses reflection — getClass().getDeclaredFields(), field.isAnnotationPresent(...), field.setAccessible(true) and field.get(obj) — to check the @Positive and @MaxLength rules and return the error messages as a String[]. 5. In main, create an Account with a negative balance and run AnnotationValidator.validate() to print the error.
	Continue from Practical 6. Replace matching files and add new files. The validator walks parent classes because balance is declared in Account, not SavingsAccount. javac -d out model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab07 Commit and tag lab-07.
	model/annotation/Id.java
Code	package model.annotation; import java.lang.annotation.*; @Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME) public @interface Id {}
	model/annotation/Positive.java
Code	package model.annotation; import java.lang.annotation.*; @Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME) public @interface Positive { String message() default "must be > 0"; }
	model/annotation/MaxLength.java
Code	package model.annotation; import java.lang.annotation.*; @Target(ElementType.FIELD) @Retention(RetentionPolicy.RUNTIME) public @interface MaxLength { int value(); }
	model/Account.java

Code	package model;
	import model.annotation.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;

```
    }
    protected Account(Account other) {
        accountNumber = other.accountNumber;
        ownerName = other.ownerName; balance = other.balance; active = other.active;
    }
    @Override
    public String toString() {
        return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
    }
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Account a)) { return false; }
        return accountNumber.equals(a.accountNumber);
    }
    @Override
    public int hashCode() { return accountNumber.hashCode(); }
    public Account(String ownerName) { this(ownerName, 0); }
    public abstract double interestRate();
    public abstract boolean canWithdraw(long amount);
    @Override
    public void deposit(long amount) {
        if (amount > 0) { balance += amount; }
    }
    @Override
    public boolean withdraw(long amount) {
        if (amount > 0 && canWithdraw(amount)) { balance -= amount; return true; }
        return false;
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

model/Customer.java

Code

```
package model;
public class Customer implements Cloneable {
    public static class Address {
        private final String line, city, pincode;
        public Address(String line, String city, String pincode) {
            this.line = line; this.city = city; this.pincode = pincode;
        }
        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }
    }
    private Address address;
    public Customer(String name, String email, String mobile, Address address) {
        this(name,email,mobile); this.address = address;
    }
    public Address getAddress() { return address; }
    @Override
    public Customer clone() {
        try { return (Customer) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }
    }
    private String name, email, mobile;
    private final String customerId;
    private static long customerCounter = 100;
    private static String generateCustomerId() { return "CUST" + (++customerCounter); }
    public Customer(String name, String email, String mobile) {
        this.name = name; this.email = email; this.mobile = mobile;
        customerId = generateCustomerId();
    }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
    public String getCustomerId() { return customerId; }
```

	}
	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	@Override
	public double interestRate() { return 4.0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	@Override
	public double interestRate() { return 0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	model/FixedDepositAccount.java

Code	package model;
	public class FixedDepositAccount extends Account {
	public FixedDepositAccount(String name, long balance) { super(name,balance); }
	@Override
	public double interestRate() { return 7.0; }
	@Override
	public boolean canWithdraw(long amount) { return false; }
	}

	service/WithdrawRule.java
Code	package service; import model.Account; @FunctionalInterface public interface WithdrawRule { boolean allow(Account account, long amount); }
	util/AnnotationValidator.java

Code	package util;
	import model.annotation.*;
	import java.lang.reflect.Field;
	import java.util.ArrayList;
	public class AnnotationValidator {
	public static String[] validate(Object obj) {
	ArrayList<String> errors = new ArrayList<>();
	for (Class<?> type = obj.getClass(); type != null; type = type.getSuperclass()) {
	for (Field field : type.getDeclaredFields()) {
	if (!field.isAnnotationPresent(Positive.class)
	&& !field.isAnnotationPresent(MaxLength.class)) { continue; }
	try {
	field.setAccessible(true);
	Object value = field.get(obj);
	Positive p = field.getAnnotation(Positive.class);
	if (p != null && value instanceof Number n && n.doubleValue() <= 0) {
	errors.add(field.getName() + ": " + p.message());

```
                }
                MaxLength m = field.getAnnotation(MaxLength.class);
                if (m != null && value instanceof String s && s.length() > m.value()) {
                    errors.add(field.getName() + ": maximum length " + m.value());
                }
            } catch (IllegalAccessException e) { throw new IllegalStateException(e); }
        }
    }
    return errors.toArray(new String[0]);
  }
}
```

service/Lab07.java

Code

```
package service;
import model.*;
import util.AnnotationValidator;
public class Lab07 {
   public static void main(String[] args) {
      Account bad = new SavingsAccount("Tej",-100,0);
      for (String error : AnnotationValidator.validate(bad)) { System.out.println(error); }
      Account good = new SavingsAccount("Riya",1000,0);
      System.out.println("Valid errors: " + AnnotationValidator.validate(good).length);
   }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB07>javac -d out model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB07>java -cp out service.Lab07
balance: must be > 0
Valid errors: 0

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB07>
```

Practical 8

Part B	
1)	1. In a package named exception, create a checked base class BankException that extends Exception. Then create InsufficientFundsException, AccountNotFoundException and InvalidAmountException, each extending BankException. Give InsufficientFundsException a long field shortfall. 2. Change withdraw(long amount) to declare throws InsufficientFundsException, InvalidAmountException; throw InsufficientFundsException when the balance is too low and InvalidAmountException when the amount is less than or equal to zero. Make deposit(long amount) throw InvalidAmountException for a non-positive amount. 3. Add a method transfer(Account to, long amount) that uses try, catch and finally and re-throws a BankException when needed. 4. In main, wrap the operations in try-catch blocks and print friendly messages; use a finally block; and show one try-with-resources example using a small AutoCloseable class that you create.
	Continue from Practical 7. Replace Account and Transactable so their checked exception declarations agree. The boolean return remains for successful withdrawals; failures now throw exceptions. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab08 Commit and tag lab-08.
	exception/BankException.java

Code	package exception;
	public class BankException extends Exception {
	public BankException(String message) { super(message); }
	}

	exception/InsufficientFundsException.java

Code	package exception;
	public class InsufficientFundsException extends BankException {
	private long shortfall;
	public InsufficientFundsException(long shortfall) {
	super("Withdrawal failed: short by " + shortfall); this.shortfall = shortfall;
	}
	public long getShortfall() { return shortfall; }
	}

	exception/InvalidAmountException.java

Code	package exception;
	public class InvalidAmountException extends BankException {
	public InvalidAmountException() { super("Amount must be positive"); }
	}

	exception/AccountNotFoundException.java

Code	package exception;
	public class AccountNotFoundException extends BankException {
	public AccountNotFoundException(String id) { super("Account not found: " + id); }
	}

	model/Transactable.java

Code	package model;
	import exception.*;
	public interface Transactable {
	void deposit(long amount) throws InvalidAmountException;
	boolean withdraw(long amount) throws InsufficientFundsException, InvalidAmountException;

	}
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	balance += amount;
	}
	@Override
	public boolean withdraw(long amount)
	throws InsufficientFundsException, InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	if (!canWithdraw(amount)) {
	long available = Math.max(0, withdrawalCapacity());
	throw new InsufficientFundsException(Math.max(0,amount - available));
	}
	balance -= amount; return true;
	}
	public void transfer(Account to, long amount) throws BankException {
	try {
	if (to == null) { throw new AccountNotFoundException("destination"); }
	if (to == this) { throw new BankException("Choose a different account"); }
	withdraw(amount); to.deposit(amount);
	} catch (BankException e) {
	System.out.println("Transfer failed: " + e.getMessage()); throw e;
	} finally { System.out.println("Transfer attempt completed"); }
	}
	public String getAccountNumber() { return accountNumber; }
	public String getOwnerName() { return ownerName; }
	@Override
	public long getBalance() { return balance; }
	public boolean isActive() { return active; }
	}

	service/Lab08.java

Code	package service;
	import model.*;
	import exception.*;
	class BankSession implements AutoCloseable {
	public void close() { System.out.println("Session closed"); }
	}
	public class Lab08 {
	public static void main(String[] args) {
	Account a = new SavingsAccount("Tej",1000,0);
	Account b = new SavingsAccount("Riya",500,0);
	try (BankSession session = new BankSession()) {
	try { a.withdraw(5000); }
	catch (InsufficientFundsException e) { System.out.println(e.getMessage()); }
	try { a.deposit(0); }
	catch (InvalidAmountException e) { System.out.println(e.getMessage()); }
	try { a.transfer(null,100); }
	catch (BankException e) { System.out.println(e.getMessage()); }
	a.transfer(b,200);
	System.out.println(a); System.out.println(b);
	} catch (BankException e) { System.out.println(e.getMessage()); }
	finally { System.out.println("Bank operations completed"); }
	}
	}

	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() - minBalance); }
	@Override
	public double interestRate() { return 4.0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() + overdraftLimit); }
	@Override
	public double interestRate() { return 0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() + overdraftLimit;
	}
	}

	model/FixedDepositAccount.java
Code	package model; public class FixedDepositAccount extends Account { public FixedDepositAccount(String name, long balance) { super(name,balance); } @Override protected long withdrawalCapacity() { return 0; } @Override public double interestRate() { return 7.0; }

```
    @Override
    public boolean canWithdraw(long amount) { return false; }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB08>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB08>java -cp out service.Lab08
Withdrawal failed: short by 4000
Amount must be positive
Transfer failed: Account not found: destination
Transfer attempt completed
Account not found: destination
Transfer attempt completed
AC0001 | Tej | 800 | true
AC0002 | Riya | 700 | true
Session closed
Bank operations completed

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB08>
```

Practical 9

Part B	
1)	1. Create a class named AccountWorker that implements Runnable with fields Account account, int times and long amount; in run(), loop times times and call account.deposit(amount). 2. In main, create one account, start about 10 threads using AccountWorker, call join() on each, then print the final balance — it will be wrong because of the race condition. 3. Make deposit and withdraw synchronized (or use a synchronized block on a private lock object) and run the program again — the balance is now correct. 4. Print the thread names to observe the thread lifecycle.
	First add AccountWorker and Lab09 while keeping the unsynchronized Account from Practical 8. Compile and run to observe a possible wrong balance; race results vary and may occasionally be correct. Then replace Account.java with the synchronized version below and repeat; the final balance must be 10000. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab09 Commit and tag lab-09.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public synchronized void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	balance += amount;
	}
	@Override

```
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            withdraw(amount); to.deposit(amount);
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage()); throw e;
        } finally { System.out.println("Transfer attempt completed"); }
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

service/AccountWorker.java

Code

```
package service;
import model.Account;
import exception.InvalidAmountException;
public class AccountWorker implements Runnable {
    private Account account;
    private int times;
    private long amount;
    public AccountWorker(Account account, int times, long amount) {
        this.account = account; this.times = times; this.amount = amount;
    }
    public void run() {
        System.out.println(Thread.currentThread().getName() + " started");
        try { for (int i = 0; i < times; i++) { account.deposit(amount); } }
        catch (InvalidAmountException e) { System.out.println(e.getMessage()); }
        System.out.println(Thread.currentThread().getName() + " finished");
    }
}
```

service/Lab09.java

Code

```
package service;
import model.*;
public class Lab09 {
    public static void main(String[] args) throws Exception {
        Account a = new SavingsAccount("Tej",0,0);
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++) {
            threads[i] = new Thread(new AccountWorker(a,1000,1),"Worker-" + i);
            threads[i].start();
        }
        for (Thread t : threads) { t.join(); }
        System.out.println("Final balance: " + a.getBalance());
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB09>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB09>java -cp out service.Lab09
Worker-4 started
Worker-7 started
Worker-6 started
Worker-2 started
Worker-6 finished
Worker-2 finished
Worker-3 started
Worker-5 started
Worker-3 finished
Worker-5 finished
Worker-4 finished
Worker-8 started
Worker-7 finished
Worker-8 finished
Worker-9 started
Worker-1 started
Worker-9 finished
Worker-0 started
Worker-1 finished
Worker-0 finished
Final balance: 10000

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB09>
```

Practical 10

Part B	
1)	1. Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls execute(), and a method stop() that calls shutdown() and awaitTermination(). 2. Create Runnable tasks that each perform a deposit or withdrawal; submit many of them, then stop the processor and wait for completion. 3. Build a simple producer-consumer: one thread adds transactions to a shared array buffer and another thread processes them, using wait() and notify() inside synchronized blocks. 4. Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first.
	Continue from Practical 9. Run service.DeadlockDemo separately to demonstrate the deliberate hang; stop with Ctrl+C. Run service.Lab10 for the fixed account-number lock order, thread pool and buffer. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.DeadlockDemo java -cp out service.Lab10 Commit and tag lab-10.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing {
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public synchronized void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }
	balance += amount;

```
    }
    @Override
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            Account first = this;
            Account second = to;
            if (accountNumber.compareTo(to.accountNumber) > 0) {
                first = to; second = this;
            }
            synchronized (first) {
                synchronized (second) { withdraw(amount); to.deposit(amount); }
            }
        } catch (BankException e) { throw e; }
        finally { System.out.println("Transfer attempt completed"); }
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

service/TransactionProcessor.java

Code

```
package service;
import java.util.concurrent.*;
public class TransactionProcessor {
    private ExecutorService pool = Executors.newFixedThreadPool(4);
    public void submit(Runnable task) { pool.execute(task); }
    public void stop() throws InterruptedException {
        pool.shutdown();
        if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
            pool.shutdownNow();
            if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
                throw new IllegalStateException("Tasks did not stop");
            }
        }
    }
}
```

service/TransactionBuffer.java

Code

```
package service;
public class TransactionBuffer {
    private Runnable[] buffer = new Runnable[3];
    private int head, tail, size;
    public synchronized void put(Runnable task) throws InterruptedException {
        while (size == buffer.length) { wait(); }
        buffer[tail] = task; tail = (tail + 1) % buffer.length; size++; notify();
    }
    public synchronized Runnable take() throws InterruptedException {
        while (size == 0) { wait(); }
        Runnable task = buffer[head]; buffer[head] = null;
        head = (head + 1) % buffer.length; size--; notify(); return task;
    }
}
```

	service/Lab10.java

Code	package service;
	import model.*;
	import exception.*;
	public class Lab10 {
	  public static void main(String[] args) throws Exception {
	  Account a = new SavingsAccount("Tej",10000,0);
	  Account b = new SavingsAccount("Riya",10000,0);
	  TransactionProcessor processor = new TransactionProcessor();
	  for (int i = 0; i < 10; i++) {
	  processor.submit(() -> {
	  try { a.deposit(100); a.withdraw(50); }
	  catch (BankException e) { System.out.println(e.getMessage()); }
	  });
	  }
	  processor.stop();
	  TransactionBuffer buffer = new TransactionBuffer();
	  Thread producer = new Thread(() -> {
	  try {
	  for (int i = 0; i < 5; i++) {
	  buffer.put(() -> {
	  try { a.deposit(10); }
	  catch (BankException e) { System.out.println(e.getMessage()); }
	  });
	  }
	  } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
	  });
	  Thread consumer = new Thread(() -> {
	  try { for (int i = 0; i < 5; i++) { buffer.take().run(); } }
	  catch (InterruptedException e) { Thread.currentThread().interrupt(); }
	  });
	  producer.start(); consumer.start(); producer.join(); consumer.join();
	  Thread t1 = new Thread(() -> move(a,b));
	  Thread t2 = new Thread(() -> move(b,a));
	  t1.start(); t2.start(); t1.join(); t2.join();
	  System.out.println(a); System.out.println(b);
	  }
	  static void move(Account from, Account to) {
	  try { from.transfer(to,100); }
	  catch (BankException e) { System.out.println(e.getMessage()); }
	  }
	}

	service/DeadlockDemo.java

Code	package service;
	import model.*;
	import java.util.concurrent.CountDownLatch;
	public class DeadlockDemo {
	  public static void main(String[] args) {
	  Account a = new SavingsAccount("Tej",1000,0);
	  Account b = new SavingsAccount("Riya",1000,0);
	  CountDownLatch held = new CountDownLatch(2);
	  new Thread(() -> badTransfer(a,b,held)).start();
	  new Thread(() -> badTransfer(b,a,held)).start();
	  }
	  static void badTransfer(Account from, Account to, CountDownLatch held) {
	  synchronized (from) {
	  held.countDown();
	  try { held.await(); }
	  catch (InterruptedException e) { Thread.currentThread().interrupt(); return; }
	  synchronized (to) {
	  try { from.withdraw(100); to.deposit(100); }
	  catch (Exception e) { System.out.println(e.getMessage()); }
	  }
	  }
	  }
	}

Output	

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB10>javac -d out exception*.java model\annotation*.java model*.java util*.java service*.java
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB10>java -cp out service.Lab10
Transfer attempt completed
Transfer attempt completed
AC0001 | Tej | 10550 | true
AC0002 | Riya | 10000 | true
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB10>

Practical 11

Part B	
1)	1. Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field. 2. Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream. 3. Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND. 4. Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds up the total deposits and total withdrawals in long variables; also read file details (size, last-modified) using Files.readAttributes, then write the summary to a report file. 5. In main, save the accounts, reload them into a new array and print them to confirm the data survived, then generate and print the report.
	Continue from Practical 10. Subclasses inherit Serializable. readObject restores the account-number counter after loading. Logs append, so running the demo again adds another set of log entries. Add data/ and out/ to .gitignore. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab11 Commit source only and tag lab-11.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing, java.io.Serializable {
	private static final long serialVersionUID = 1L;
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);

```
    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        balance += amount;
    }
    @Override
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            Account first = this;
            Account second = to;
            if (accountNumber.compareTo(to.accountNumber) > 0) {
                first = to; second = this;
            }
            synchronized (first) {
                synchronized (second) { withdraw(amount); to.deposit(amount); }
            }
        } catch (BankException e) { throw e; }
        finally { System.out.println("Transfer attempt completed"); }
    }
    private void readObject(java.io.ObjectInputStream in)
        throws java.io.IOException, ClassNotFoundException {
        in.defaultReadObject();
        long number = Long.parseLong(accountNumber.substring(2));
        synchronized (Account.class) { counter = Math.max(counter,number); }
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

model/Customer.java

Code

```
package model;
public class Customer implements Cloneable, java.io.Serializable {
    private static final long serialVersionUID = 1L;
    public static class Address implements java.io.Serializable {
        private static final long serialVersionUID = 1L;
        private final String line, city, pincode;
        public Address(String line, String city, String pincode) {
            this.line = line; this.city = city; this.pincode = pincode;
        }
        public String getLine() { return line; }
        public String getCity() { return city; }
        public String getPincode() { return pincode; }
    }
    private Address address;
    public Customer(String name, String email, String mobile, Address address) {
        this(name,email,mobile); this.address = address;
    }
    public Address getAddress() { return address; }
    @Override
    public Customer clone() {
        try { return (Customer) super.clone(); }
        catch (CloneNotSupportedException e) { throw new AssertionError(e); }
    }
    private String name, email, mobile;
    private final String customerId;
```

```
    private static long customerCounter = 100;
    private static String generateCustomerId() { return "CUST" + (++customerCounter); }
    public Customer(String name, String email, String mobile) {
        this.name = name; this.email = email; this.mobile = mobile;
        customerId = generateCustomerId();
    }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getMobile() { return mobile; }
    public String getCustomerId() { return customerId; }
}
```

util/StatePersister.java

Code

```
package util;
import model.Account;
import java.io.*;
import java.nio.file.*;
public class StatePersister {
    public static void save(Account[] accounts, Path file) throws IOException {
        Path parent = file.toAbsolutePath().getParent();
        Files.createDirectories(parent);
        try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(accounts);
        }
    }
    public static Account[] load(Path file) throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
            return (Account[]) in.readObject();
        }
    }
}
```

util/TransactionLog.java

Code

```
package util;
import java.nio.file.*;
import java.io.IOException;
public class TransactionLog {
    public static synchronized void append(Path file, String line) throws IOException {
        Files.createDirectories(file.toAbsolutePath().getParent());
        Files.writeString(file,line + System.lineSeparator(),
            StandardOpenOption.CREATE,StandardOpenOption.APPEND);
    }
}
```

util/ReportGenerator.java

Code

```
package util;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.io.*;
public class ReportGenerator {
    public static String generate(Path folder, Path reportFile) throws IOException {
        Files.createDirectories(folder);
        long deposits = 0, withdrawals = 0;
        StringBuilder details = new StringBuilder();
        try (DirectoryStream<Path> files = Files.newDirectoryStream(folder,"*.log")) {
            for (Path file : files) {
                if (!Files.isRegularFile(file) || Files.size(file) == 0) { continue; }
                BasicFileAttributes attr = Files.readAttributes(file,BasicFileAttributes.class);
                details.append(file.getFileName()).append(" Size: ").append(attr.size())
                    .append(" Modified: ").append(attr.lastModifiedTime()).append("\n");
                try (BufferedReader reader = Files.newBufferedReader(file)) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        String[] p = line.trim().split("\\s+");
                        if (p.length != 3) { continue; }
                        try {
                            long amount = Long.parseLong(p[2]);
                            if (amount <= 0) { continue; }
```

	if (p[0].equals("DEPOSIT")) { deposits += amount; } else if (p[0].equals("WITHDRAW")) { withdrawals += amount; } } catch (NumberFormatException e) { continue; } } } } } details.append("Deposits: ").append(deposits).append("\nWithdrawals: ") .append(withdrawals).append("\nNet change: ").append(deposits - withdrawals); Files.createDirectories(reportFile.toAbsolutePath().getParent()); Files.writeString(reportFile,details); return details.toString(); } }
	service/Lab11.java

Code	package service;
	import model.*;
	import util.*;
	import java.nio.file.*;
	public class Lab11 {
	public static void main(String[] args) throws Exception {
	Account[] accounts = {new SavingsAccount("Tej",5000,0),
	new CurrentAccount("Riya",2000,1000),new FixedDepositAccount("Aman",9000)};
	accounts[0].deposit(500); accounts[1].withdraw(200);
	Path data = Path.of("data/accounts.dat");
	StatePersister.save(accounts,data);
	Account[] loaded = StatePersister.load(data);
	for (Account a : loaded) { System.out.println(a); }
	Path logs = Path.of("data/logs");
	TransactionLog.append(logs.resolve("daily.log"),"DEPOSIT "
	+ accounts[0].getAccountNumber() + " 500");
	TransactionLog.append(logs.resolve("daily.log"),"WITHDRAW "
	+ accounts[1].getAccountNumber() + " 200");
	System.out.println(ReportGenerator.generate(logs,Path.of("data/report.txt")));
	}
	}

	model/SavingsAccount.java

Code	package model;
	public class SavingsAccount extends Account implements Premium {
	private static final long serialVersionUID = 1L;
	private long minBalance;
	public SavingsAccount(String name, long balance, long minBalance) {
	super(name,balance); this.minBalance = minBalance;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() - minBalance); }
	@Override
	public double interestRate() { return 4.0; }
	@Override
	public boolean canWithdraw(long amount) {
	return amount > 0 && amount <= getBalance() - minBalance;
	}
	}

	model/CurrentAccount.java

Code	package model;
	public class CurrentAccount extends Account {
	private static final long serialVersionUID = 1L;
	private long overdraftLimit;
	public CurrentAccount(String name, long balance, long overdraftLimit) {
	super(name,balance); this.overdraftLimit = overdraftLimit;
	}
	@Override
	protected long withdrawalCapacity() { return Math.max(0,getBalance() + overdraftLimit); }
	@Override
	public double interestRate() { return 0; }
	@Override

```
    public boolean canWithdraw(long amount) {
        return amount > 0 && amount <= getBalance() + overdraftLimit;
    }
}
```

model/FixedDepositAccount.java

Code

```
package model;
public class FixedDepositAccount extends Account {
    private static final long serialVersionUID = 1L;
    public FixedDepositAccount(String name, long balance) { super(name,balance); }
    @Override
    protected long withdrawalCapacity() { return 0; }
    @Override
    public double interestRate() { return 7.0; }
    @Override
    public boolean canWithdraw(long amount) { return false; }
}
```

Output



Practical 12

Part B	
1)	1. Create a class named AccountStore that keeps accounts in a HashMap<String, Account> using the account number as the key. Add methods add(Account account), get(String accountNumber) and getAll(). 2. Keep a transaction history in an ArrayList<String>. 3. Add a method sortedByBalance() that returns an ArrayList<Account> sorted with a Comparator. 4. Make Account implement Comparable<Account> by comparing account numbers in compareTo(). 5. In the TransactionProcessor, use a ConcurrentHashMap for the shared accounts so it is thread-safe. 6. In main, add several accounts to the store and print them sorted by balance.
	Continue from Practical 11. AccountStore uses HashMap and ArrayList. TransactionProcessor uses ConcurrentHashMap while Account methods still synchronize balance updates. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab12 Commit and tag lab-12.
	model/Account.java

Code	package model;
	import model.annotation.*;
	import exception.*;
	public abstract class Account implements Transactable, InterestBearing, java.io.Serializable, Comparable<Account> {
	private static final long serialVersionUID = 1L;
	@Id
	private final String accountNumber;
	@MaxLength(30)
	private String ownerName;
	@Positive
	private long balance;
	private boolean active = true;
	private static long counter;
	public Account(String ownerName, long balance) {
	accountNumber = String.format("AC%04d", ++counter);
	this.ownerName = ownerName; this.balance = balance;
	}
	protected Account(Account other) {
	accountNumber = other.accountNumber;
	ownerName = other.ownerName; balance = other.balance; active = other.active;
	}
	@Override
	public String toString() {
	return accountNumber + " | " + ownerName + " | " + balance + " | " + active;
	}
	@Override
	public boolean equals(Object o) {
	if (!(o instanceof Account a)) { return false; }
	return accountNumber.equals(a.accountNumber);
	}
	@Override
	public int hashCode() { return accountNumber.hashCode(); }
	public Account(String ownerName) { this(ownerName, 0); }
	protected long withdrawalCapacity() { return balance; }
	public abstract double interestRate();
	public abstract boolean canWithdraw(long amount);
	@Override
	public synchronized void deposit(long amount) throws InvalidAmountException {
	if (amount <= 0) { throw new InvalidAmountException(); }

```
        balance += amount;
    }
    @Override
    public synchronized boolean withdraw(long amount)
        throws InsufficientFundsException, InvalidAmountException {
        if (amount <= 0) { throw new InvalidAmountException(); }
        if (!canWithdraw(amount)) {
            long available = Math.max(0, withdrawalCapacity());
            throw new InsufficientFundsException(Math.max(0,amount - available));
        }
        balance -= amount; return true;
    }
    public void transfer(Account to, long amount) throws BankException {
        try {
            if (to == null) { throw new AccountNotFoundException("destination"); }
            if (to == this) { throw new BankException("Choose a different account"); }
            Account first = this;
            Account second = to;
            if (accountNumber.compareTo(to.accountNumber) > 0) {
                first = to; second = this;
            }
            synchronized (first) {
                synchronized (second) { withdraw(amount); to.deposit(amount); }
            }
        } catch (BankException e) { throw e; }
        finally { System.out.println("Transfer attempt completed"); }
    }
    private void readObject(java.io.ObjectInputStream in)
        throws java.io.IOException, ClassNotFoundException {
        in.defaultReadObject();
        long number = Long.parseLong(accountNumber.substring(2));
        synchronized (Account.class) { counter = Math.max(counter,number); }
    }
    @Override
    public int compareTo(Account other) {
        return accountNumber.compareTo(other.accountNumber);
    }
    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName() { return ownerName; }
    @Override
    public synchronized long getBalance() { return balance; }
    public boolean isActive() { return active; }
}
```

service/AccountStore.java

Code

```
package service;
import model.Account;
import java.util.*;
public class AccountStore {
    private HashMap<String,Account> accounts = new HashMap<>();
    private ArrayList<String> history = new ArrayList<>();
    public void add(Account account) { accounts.put(account.getAccountNumber(),account); }
    public Account get(String number) { return accounts.get(number); }
    public ArrayList<Account> getAll() { return new ArrayList<>(accounts.values()); }
    public void record(String line) { history.add(line); }
    public ArrayList<String> getHistory() { return new ArrayList<>(history); }
    public ArrayList<Account> sortedByBalance() {
        ArrayList<Account> list = getAll();
        list.sort(Comparator.comparingLong(Account::getBalance).reversed()); return list;
    }
}
```

service/TransactionProcessor.java

Code

```
package service;
import java.util.concurrent.*;
import model.Account;
import java.util.Collection;
public class TransactionProcessor {
```

```
    private ConcurrentHashMap<String,Account> accounts = new ConcurrentHashMap<>();
    public void add(Account a) { accounts.put(a.getAccountNumber(),a); }
    public Account get(String number) { return accounts.get(number); }
    public void useAccounts(Collection<Account> source) {
        accounts.clear();
        for (Account a : source) { add(a); }
    }
    private ExecutorService pool = Executors.newFixedThreadPool(4);
    public void submit(Runnable task) { pool.execute(task); }
    public void stop() throws InterruptedException {
        pool.shutdown();
        if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
            pool.shutdownNow();
            if (!pool.awaitTermination(1,TimeUnit.MINUTES)) {
                throw new IllegalStateException("Tasks did not stop");
            }
        }
    }
}
```

service/Lab12.java

Code

```
package service;
import model.*;
public class Lab12 {
    public static void main(String[] args) throws Exception {
        AccountStore store = new AccountStore();
        Account a = new SavingsAccount("Tej",5000,0);
        Account b = new SavingsAccount("Riya",2000,0);
        Account c = new SavingsAccount("Aman",9000,0);
        store.add(a); store.add(b); store.add(c);
        store.record("Opened " + a.getAccountNumber());
        System.out.println(store.get(b.getAccountNumber()));
        for (Account item : store.sortedByBalance()) { System.out.println(item); }
        System.out.println(store.getHistory());
        TransactionProcessor processor = new TransactionProcessor();
        processor.useAccounts(store.getAll());
        for (int i = 0; i < 10; i++) {
            processor.submit(new AccountWorker(processor.get(a.getAccountNumber()),100,1));
        }
        processor.stop(); System.out.println(a);
    }
}
```

Output

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB12>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB12>java -cp out service.Lab12
AC0002 | Riya | 2000 | true
AC0003 | Aman | 9000 | true
AC0001 | Tej | 5000 | true
AC0002 | Riya | 2000 | true
[Opened AC0001]
pool-1-thread-2 started
pool-1-thread-4 started
pool-1-thread-2 finished
pool-1-thread-1 started
pool-1-thread-2 started
pool-1-thread-2 finished
pool-1-thread-3 started
pool-1-thread-4 finished
pool-1-thread-1 finished
pool-1-thread-4 started
pool-1-thread-1 started
pool-1-thread-4 finished
pool-1-thread-1 finished
pool-1-thread-2 started
pool-1-thread-2 finished
pool-1-thread-3 finished
pool-1-thread-4 started
pool-1-thread-1 started
pool-1-thread-1 finished
pool-1-thread-4 finished
AC0001 | Tej | 6000 | true

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB12>
```

Practical 13

Part B	
1)	1. Create a generic class named Repository<T> that holds a HashMap<String, T>. Add methods T save(T entity), T findById(String id), ArrayList<T> findAll() and boolean delete(String id). 2. Inside save(), find the id by reflection: look for the field marked @Id (from Practical 7) and read its value as the map key. 3. Add a method marked @SafeVarargs called saveAll(T... items) that saves several objects at once. 4. Add one generic method with a bounded type parameter, for example static <N extends Number> double average(N[] values). 5. In main, create a Repository<Account>, save several accounts, fetch one by id, and print all of them.
	Continue from Practical 12. Repository scans inherited fields so the @Id on Account works for every subtype. Customer also gets @Id for reuse. Synchronized repository methods protect its HashMap during later batch access. javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java java -cp out service.Lab13 Commit and tag lab-13.
	service/Repository.java

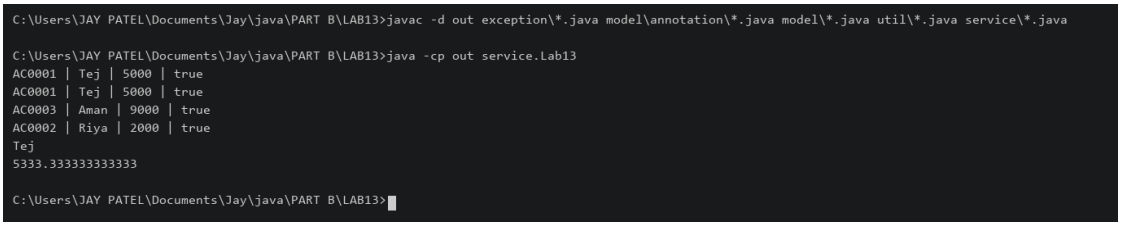
Code	package service;
	import model.annotation.Id;
	import java.lang.reflect.Field;
	import java.util.*;
	public class Repository<T> {
	private HashMap<String,T> data = new HashMap<>();
	private String idOf(T entity) {
	for (Class<?> type = entity.getClass(); type != null; type = type.getSuperclass()) {
	for (Field field : type.getDeclaredFields()) {
	if (field.isAnnotationPresent(Id.class)) {
	try {
	field.setAccessible(true);
	Object value = field.get(entity);
	if (value == null) { throw new IllegalArgumentException("Null id"); }
	return value.toString();
	} catch (IllegalAccessException e) { throw new IllegalStateException(e); }
	}
	}
	}
	throw new IllegalArgumentException("No @Id field");
	}
	public synchronized T save(T entity) { data.put(idOf(entity),entity); return entity; }
	public synchronized T findById(String id) { return data.get(id); }
	public synchronized ArrayList<T> findAll() { return new ArrayList<>(data.values()); }
	public synchronized boolean delete(String id) { return data.remove(id) != null; }
	@SafeVarargs
	public final synchronized void saveAll(T... items) {
	for (T item : items) { save(item); }
	}
	public static <N extends Number> double average(N[] values) {
	if (values.length == 0) { throw new IllegalArgumentException("Empty array"); }
	double total = 0;
	for (N n : values) { total += n.doubleValue(); }
	return total / values.length;
	}
	}

	model/Customer.java

Code	package model;
	public class Customer implements Cloneable, java.io.Serializable {
	private static final long serialVersionUID = 1L;
	public static class Address implements java.io.Serializable {
	private static final long serialVersionUID = 1L;
	private final String line, city, pincode;
	public Address(String line, String city, String pincode) {
	this.line = line; this.city = city; this.pincode = pincode;
	}
	public String getLine() { return line; }
	public String getCity() { return city; }
	public String getPincode() { return pincode; }
	}
	private Address address;
	public Customer(String name, String email, String mobile, Address address) {
	this(name,email,mobile); this.address = address;
	}
	public Address getAddress() { return address; }
	@Override
	public Customer clone() {
	try { return (Customer) super.clone(); }
	catch (CloneNotSupportedException e) { throw new AssertionError(e); }
	}
	private String name, email, mobile;
	@model.annotation.Id
	private final String customerId;
	private static long customerCounter = 100;
	private static String generateCustomerId() { return "CUST" + (++customerCounter); }
	public Customer(String name, String email, String mobile) {
	this.name = name; this.email = email; this.mobile = mobile;
	customerId = generateCustomerId();
	}
	public String getName() { return name; }
	public String getEmail() { return email; }
	public String getMobile() { return mobile; }
	public String getCustomerId() { return customerId; }
	}

	service/Lab13.java

Code	package service;
	import model.*;
	public class Lab13 {
	public static void main(String[] args) {
	Repository<Account> accounts = new Repository<>();
	Account a = new SavingsAccount("Tej",5000,0);
	Account b = new CurrentAccount("Riya",2000,1000);
	Account c = new FixedDepositAccount("Aman",9000);
	accounts.saveAll(a,b,c);
	System.out.println(accounts.findById(a.getAccountNumber()));
	for (Account item : accounts.findAll()) { System.out.println(item); }
	Repository<Customer> customers = new Repository<>();
	Customer customer = new Customer("Tej","tej@test.com","9876543210");
	customers.save(customer);
	System.out.println(customers.findById(customer.getCustomerId()).getName());
	System.out.println(Repository.average(new Long[]{5000L,2000L,9000L}));
	}
	}

Output	

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB13>javac -d out exception*.java model\annotation*.java model*.java util*.java service*.java
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB13>java -cp out service.Lab13
AC0001 | Tej | 5000 | true
AC0001 | Tej | 5000 | true
AC0003 | Aman | 9000 | true
AC0002 | Riya | 2000 | true
Tej
5333.333333333333
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB13>

Practical 14

Part B	
1)	1. Create a class named BankService that uses Repository<Account>, the Validator and the exception classes to provide openAccount, deposit, withdraw, transfer, statement and listAccounts. 2. Connect the Practical 1 menu so each option calls the matching BankService method; load saved accounts on start-up (Practical 11) and save them on exit. 3. Use the TransactionProcessor (Practical 10) over the thread-safe store (Practical 12) to process a batch of transactions safely, and add the report (Practical 11) as a menu option. 4. Write a few JUnit tests using @Test: a deposit increases the balance; a withdrawal over the limit throws an exception; many concurrent deposits keep the balance correct. 5. (Optional) Add an @Audit annotation and a reflective logger that records which service method was called. 6. Build the final runnable JAR and complete the README.
	Continue from Practical 13. Add BankService and the test file; replace service/MiniBank.java. Keep all other current package files. Build, run and test using the README commands below. Data is saved when Exit is selected. The JUnit tests require the external JUnit JAR.
	service/BankService.java

Code	package service;
	import model.*;
	import util.*;
	import exception.*;
	import java.nio.file.*;
	import java.io.IOException;
	import java.util.*;
	public class BankService {
	private Repository<Account> repository = new Repository<>();
	private ArrayList<String> history = new ArrayList<>();
	private Path folder;
	public BankService(Path folder) throws IOException, ClassNotFoundException {
	this.folder = folder;
	Files.createDirectories(folder.resolve("logs"));
	Path state = folder.resolve("accounts.dat");
	if (Files.exists(state)) { repository.saveAll(StatePersister.load(state)); }
	Path log = folder.resolve("logs/daily.log");
	if (Files.exists(log)) { history.addAll(Files.readAllLines(log)); }
	}
	private Account require(String number) throws AccountNotFoundException {
	Account account = repository.findById(number);
	if (account == null) { throw new AccountNotFoundException(number); }
	return account;
	}
	private void record(String type, String id, long amount) throws IOException {
	String line = type + " " + id + " " + amount;
	TransactionLog.append(folder.resolve("logs/daily.log"),line);
	history.add(line);
	}
	public synchronized Account openAccount(String type, String name, String email,
	String mobile, String pan, String ifsc, long opening) throws BankException {
	if (name == null || name.isBlank() || name.length() > 30
	|| !Validator.isValidEmail(email) || !Validator.isValidMobile(mobile)
	|| !Validator.isValidPan(pan) || !Validator.isValidIfsc(ifsc)) {
	throw new BankException("Invalid customer details");
	}
	if (opening <= 0) { throw new InvalidAmountException(); }
	Account account;
	switch (type.toUpperCase()) {
	case "SAVINGS" -> account = new SavingsAccount(name,opening,0);
	case "CURRENT" -> account = new CurrentAccount(name,opening,1000);

```
            case "FD" -> account = new FixedDepositAccount(name,opening);
            default -> throw new BankException("Invalid account type");
        }
        String[] errors = AnnotationValidator.validate(account);
        if (errors.length > 0) { throw new BankException(String.join(", ",errors)); }
        repository.save(account); return account;
    }
    public synchronized void deposit(String id, long amount) throws BankException, IOException {
        require(id).deposit(amount); record("DEPOSIT",id,amount);
    }
    public synchronized void withdraw(String id, long amount) throws BankException, IOException {
        require(id).withdraw(amount); record("WITHDRAW",id,amount);
    }
    public synchronized void transfer(String from, String to, long amount)
        throws BankException, IOException {
        Account source = require(from), target = require(to);
        source.transfer(target,amount);
        record("WITHDRAW",from,amount); record("DEPOSIT",to,amount);
    }
    public synchronized String statement(String id) throws BankException {
        StringBuilder result = new StringBuilder(StatementFormatter.buildStatement(require(id)));
        result.append("\nTransactions\n");
        for (String line : history) {
            String[] p = line.split(" ");
            if (p.length == 3 && p[1].equals(id)) { result.append(line).append("\n"); }
        }
        return result.toString();
    }
    public synchronized ArrayList<Account> listAccounts() {
        ArrayList<Account> list = repository.findAll();
        Collections.sort(list); return list;
    }
    public synchronized long balance(String id) throws BankException { return require(id).getBalance();
}
    public synchronized void save() throws IOException {
        StatePersister.save(repository.findAll().toArray(new Account[0]),folder.resolve("accounts.dat"));
    }
    public synchronized String report() throws IOException {
        return ReportGenerator.generate(folder.resolve("logs"),folder.resolve("report.txt"));
    }
    public void batchDeposit(String id, int tasks, long amount) throws Exception {
        if (tasks <= 0 || amount <= 0) { throw new InvalidAmountException(); }
        TransactionProcessor processor = new TransactionProcessor();
        processor.useAccounts(listAccounts());
        if (processor.get(id) == null) {
            processor.stop(); throw new AccountNotFoundException(id);
        }
        java.util.concurrent.ConcurrentLinkedQueue<Exception> errors =
            new java.util.concurrent.ConcurrentLinkedQueue<>();
        for (int i = 0; i < tasks; i++) {
            processor.submit(() -> {
                try { deposit(processor.get(id).getAccountNumber(),amount); }
                catch (Exception e) { errors.add(e); }
            });
        }
        processor.stop();
        if (!errors.isEmpty()) { throw errors.peek(); }
    }
}
```

service/MiniBank.java

Code

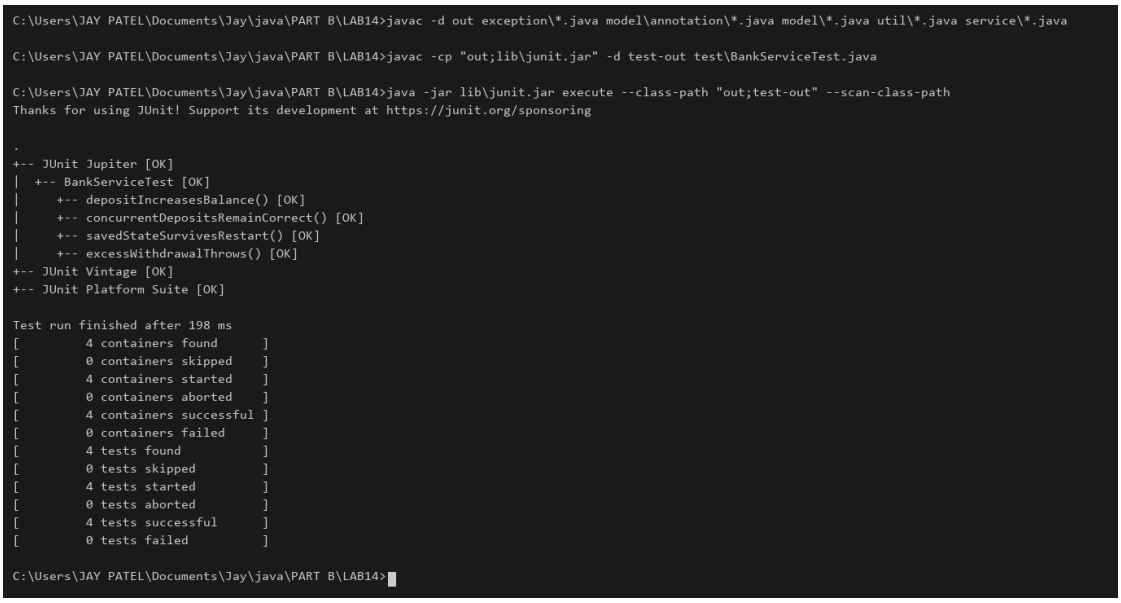
```
package service;
import model.Account;
import model.Command;
import util.*;
import java.nio.file.Path;
import java.util.Scanner;
record BankInfo(String name, String branch) {}
enum MenuOption { OPEN_ACCOUNT, DEPOSIT, WITHDRAW, TRANSFER,
```

```
            STATEMENT, LIST_ACCOUNTS, BATCH, REPORT, EXIT }
public class MiniBank {
   static String read(Scanner sc, String label) {
      System.out.print(label + ": "); return sc.nextLine().trim();
   }
   public static void main(String[] args) {
      System.out.println(new BankInfo("MiniBank","Anand"));
      try {
         BankService bank = new BankService(Path.of("data"));
         Scanner sc = new Scanner(System.in);
         boolean exit = false;
         while (!exit) {
            System.out.println("1. Open account\n2. Deposit\n3. Withdraw\n4. Transfer"
               + "\n5. Statement\n6. List accounts\n7. Batch deposits\n8. Report\n9. Exit");
            try {
               int choice = Integer.parseInt(sc.nextLine());
               if (choice < 1 || choice > 9) { System.out.println("Invalid choice"); continue; }
               MenuOption option = MenuOption.values()[choice - 1];
               switch (option) {
                  case OPEN_ACCOUNT -> {
                     String type = read(sc,"Type SAVINGS/CURRENT/FD");
                     String name = read(sc,"Name");
                     String email = read(sc,"Email");
                     String mobile = read(sc,"Mobile");
                     String pan = read(sc,"PAN");
                     String ifsc = read(sc,"IFSC");
                     long opening = Long.parseLong(read(sc,"Opening balance"));
                     Account a = bank.openAccount(type,name,email,mobile,pan,ifsc,opening);
                     System.out.println("Opened: " + a);
                  }
                  case DEPOSIT, WITHDRAW -> {
                     String id = read(sc,"Account");
                     String amount = read(sc,"Amount");
                     Command c = CommandParser.parse(option.name() + " " + id + " " + amount);
                     if (option == MenuOption.DEPOSIT) {
bank.deposit(c.accountNumber(),c.amount()); }
                     else { bank.withdraw(c.accountNumber(),c.amount()); }
                     System.out.println("Balance: " + bank.balance(id));
                  }
                  case TRANSFER -> {
                     String from = read(sc,"From account");
                     String to = read(sc,"To account");
                     String amount = read(sc,"Amount");
                     Command c = CommandParser.parse("TRANSFER " + from + " " + amount);
                     bank.transfer(c.accountNumber(),to,c.amount());
                     System.out.println("Transfer completed");
                  }
                  case STATEMENT -> System.out.println(bank.statement(read(sc,"Account")));
                  case LIST_ACCOUNTS -> {
                     for (Account a : bank.listAccounts()) { System.out.println(a); }
                  }
                  case BATCH -> {
                     String id = read(sc,"Account");
                     int tasks = Integer.parseInt(read(sc,"Number of deposits"));
                     long amount = Long.parseLong(read(sc,"Amount per deposit"));
                     bank.batchDeposit(id,tasks,amount);
                     System.out.println("Balance: " + bank.balance(id));
                  }
                  case REPORT -> System.out.println(bank.report());
                  case EXIT -> {
                     bank.save(); exit = true; System.out.println("Saved. Goodbye");
                  }
               }
            } catch (Exception e) { System.out.println("Error: " + e.getMessage()); }
         }
      } catch (Exception e) { System.out.println("Startup failed: " + e.getMessage()); }
   }
}
```

	test/BankServiceTest.java

Code	package test;
	import service.BankService;
	import model.Account;
	import exception.*;
	import java.nio.file.Path;
	import org.junit.jupiter.api.Test;
	import org.junit.jupiter.api.io.TempDir;
	import static org.junit.jupiter.api.Assertions.*;
	public class BankServiceTest {
	@TempDir Path folder;
	Account open(BankService bank) throws BankException {
	return bank.openAccount("SAVINGS","Tej","tej@test.com","9876543210",
	"ABCDE1234F","SBIN0001234",1000);
	}
	@Test void depositIncreasesBalance() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank); bank.deposit(a.getAccountNumber(),500);
	assertEquals(1500,bank.balance(a.getAccountNumber()));
	}
	@Test void excessWithdrawalThrows() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank);
	assertThrows(InsufficientFundsException.class,
	() -> bank.withdraw(a.getAccountNumber(),5000));
	assertEquals(1000,bank.balance(a.getAccountNumber()));
	}
	@Test void concurrentDepositsRemainCorrect() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank); bank.batchDeposit(a.getAccountNumber(),100,10);
	assertEquals(2000,bank.balance(a.getAccountNumber()));
	}
	@Test void savedStateSurvivesRestart() throws Exception {
	BankService bank = new BankService(folder);
	Account a = open(bank); bank.deposit(a.getAccountNumber(),500); bank.save();
	BankService loaded = new BankService(folder);
	assertEquals(1500,loaded.balance(a.getAccountNumber()));
	Account next = open(loaded);
	assertNotEquals(a.getAccountNumber(),next.getAccountNumber());
	}
	}

	README.txt
Text	MiniBank requires JDK 17 or newer. Keep the package folders model, model/annotation, util, service and exception together. Retain the files from Practicals 6 to 13 and apply the additions and replacements from Practical 14. Earlier LabNN demo classes may remain; run service.MiniBank for the final application. Build: javac -d out exception/*.java model/annotation/*.java model/*.java util/*.java service/*.java jar cfm minibank.jar manifest.mf -C out . Run: java -jar minibank.jar Choose 1 to open an account. Enter SAVINGS, CURRENT or FD, a nonblank name of at most 30 characters, a valid email, a 10-digit mobile starting 6 to 9, PAN ABCDE1234F, IFSC SBIN0001234, and a positive opening balance. Savings uses a minimum balance of 0; current allows an overdraft of 1000; fixed deposit withdrawals are locked. Options 2 to 4 perform transactions. Options 5 and 6 show statements and accounts. Option 7 submits a batch of deposits to a four-thread pool. Option 8 writes a report. Option 9 saves accounts and exits. The next launch loads data/accounts.dat. Logs are appended to data/logs/daily.log. Transfer logs contain one withdrawal and one deposit, so the net change is zero. Opening balances are initial capital and are excluded from transaction totals. Report totals cover all retained log entries. JUnit tests:

	Download JUnit Platform Console Standalone 1.10.2 and save it as lib/junit.jar. Test sources stay in test/ and are not included in the application JAR. javac -cp "out;lib/junit.jar" -d test-out test/BankServiceTest.java java -jar lib/junit.jar execute --class-path "out;test-out" --scan-class-path Use a colon instead of a semicolon in classpaths on Linux/macOS. Git: Ignore out/, test-out/, data/, lib/ and *.jar. Commit the source, manifest and README. Tag the final version lab-14. Earlier stages use tags lab-01 to lab-13.
Output	

```
C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>javac -d out exception\*.java model\annotation\*.java model\*.java util\*.java service\*.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>javac -cp "out;lib\junit.jar" -d test-out test\BankServiceTest.java

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>java -jar lib\junit.jar execute --class-path "out;test-out" --scan-class-path
Thanks for using JUnit! Support its development at https://junit.org/sponsoring

.
+-- JUnit Jupiter [OK]
|  +-- BankServiceTest [OK]
|     +-- depositIncreasesBalance() [OK]
|     +-- concurrentDepositsRemainCorrect() [OK]
|     +-- savedStateSurvivesRestart() [OK]
|     +-- excessWithdrawalThrows() [OK]
+-- JUnit Vintage [OK]
+-- JUnit Platform Suite [OK]

Test run finished after 198 ms
[         4 containers found      ]
[         0 containers skipped    ]
[         4 containers started    ]
[         0 containers aborted    ]
[         4 containers successful ]
[         0 containers failed     ]
[         4 tests found           ]
[         0 tests skipped         ]
[         4 tests started         ]
[         0 tests aborted         ]
[         4 tests successful      ]
[         0 tests failed          ]

C:\Users\JAY PATEL\Documents\Jay\java\PART B\LAB14>
```